```
import os

# ─── Paths ──────────────────────────────────────────────────────────────
BASE_DIR        = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

RAW_DATA_PATH       = os.path.join(BASE_DIR, "data", "raw")
PROCESSED_DATA_PATH = os.path.join(BASE_DIR, "data", "processed")
INTERIM_DATA_PATH   = os.path.join(BASE_DIR, "data", "interim")

ARTIFACTS_PATH  = os.path.join(BASE_DIR, "artifacts")
MODELS_PATH     = os.path.join(BASE_DIR, "artifacts", "model")
SCALERS_PATH    = os.path.join(BASE_DIR, "artifacts", "scalers")
REPORTS_PATH    = os.path.join(BASE_DIR, "artifacts", "reports")
LOGS_PATH       = os.path.join(BASE_DIR, "logs")

# ─── Dataset ────────────────────────────────────────────────────────────
# All CICIDS CSV files (in the order you want them processed)
RAW_FILES = [
    "Monday-WorkingHours.pcap_ISCX.csv",
    "Tuesday-WorkingHours.pcap_ISCX.csv",
    "Wednesday-workingHours.pcap_ISCX.csv",
    "Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv",
    "Thursday-WorkingHours-Afternoon-Infilteration.pcap_ISCX.csv",
    "Friday-WorkingHours-Morning.pcap_ISCX.csv",
    "Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv",
    "Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv",
]

PROCESSED_DATA_VERSION = "v1"
PROCESSED_FILENAME = f"processed_data_{PROCESSED_DATA_VERSION}.csv"

# ─── Columns ────────────────────────────────────────────────────────────
TARGET_COLUMN = "Label"
COLUMNS_TO_DROP = ["Flow ID", " Source IP", " Destination IP",
                   " Timestamp", "Fwd Header Length.1"]

# ─── Preprocessing ──────────────────────────────────────────────────────
TEST_SIZE    = 0.2
RANDOM_STATE = 42
SCALE_FEATURES = True
```

```
# ─── Model ──────────────────────────────────────────────────────────────
MODEL_TYPE = "random_forest"   # options: "random_forest", "xgboost", "logistic"

# ─── Flags ──────────────────────────────────────────────────────────────
SAVE_PROCESSED_DATA = True
SAVE_MODEL          = True
SAVE_SCALER         = True

# ─── Debug ──────────────────────────────────────────────────────────────
DEBUG = True

# ─── Feature Engineering ────────────────────────────────────────────────
LOW_VARIANCE_THRESHOLD = 0.0
CORRELATION_THRESHOLD = 0.95

# ── Baseline training — models to train in main.py ──────────────────────
# SVM is placed last — it is the slowest on 2.8M records (~5-10 min).
MODELS_TO_RUN = [
    "random_forest",
    "xgboost",
    "logistic",
    "decision_tree",
    "svm",
]

# ── Cross-dataset evaluation — models to use in testdrift.py ────────────
# All five models are compared for drift sensitivity.
# SVM uses scaled data (already handled in cross_dataset_evaluation).
CROSS_EVAL_MODELS = [
    "random_forest",
    "xgboost",
    "logistic",
    "decision_tree",
    "svm",
]

# ── Drift metrics ───────────────────────────────────────────────────────
# Toggle which divergence metrics are computed in compute_drift().
COMPUTE_KL          = True
COMPUTE_JS          = True
COMPUTE_WASSERSTEIN = True   # Wasserstein distance (Earth Mover's Distance)
```

```
# ── Rolling reference window ──────────────────────────────────────────────────
# If True, compute_rolling_drift() compares each batch against the
# immediately preceding batch instead of all batches against Monday.
USE_ROLLING_REFERENCE = True

# ── Drift alert ───────────────────────────────────────────────────────────────
# If the drift predictor's predicted accuracy falls below this threshold,
# a drift alert is written to artifacts/reports/drift_alerts.json.
DRIFT_ALERT_THRESHOLD = 0.80

# ── SHAP ──────────────────────────────────────────────────────────────────────
SHAP_SAMPLE_SIZE    = 500    # rows sampled from drifted batch for SHAP
SHAP_TOP_N_FEATURES = 15     # number of top features to show in SHAP plot

"""
drift_predictor.py
------------------
Phase 4 of the project: the drift prediction meta-model.

This reads the saved drift_scores.json and cross_eval_accuracy.json,
joins them into a meta-dataset, trains a regression model to predict
accuracy from divergence scores, and saves the results.

Run from src/:
    python drift_predictor.py
"""

import os
import json
import numpy as np
import joblib
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, r2_score
from config import REPORTS_PATH, MODELS_PATH
from utils import get_logger, save_experiment_report
from datetime import datetime

logger = get_logger("drift_predictor")
```

```
def load_meta_dataset():
    """
    Join drift_scores.json and cross_eval_accuracy.json
    into a single meta-dataset for the drift prediction model.
    """
    drift_path = os.path.join(REPORTS_PATH, "drift_scores.json")
    eval_path  = os.path.join(REPORTS_PATH, "cross_eval_accuracy.json")

    with open(drift_path) as f:
        drift_records = json.load(f)

    with open(eval_path) as f:
        eval_records = json.load(f)

    # Build lookup: batch_name -> accuracy
    acc_lookup = {r["batch"]: r["accuracy"] for r in eval_records}

    rows = []
    for rec in drift_records:
        batch = rec["comparison_batch"]
        if batch not in acc_lookup:
            logger.warning(f"No accuracy found for: {batch}")
            continue

        rows.append({
            "batch"              : batch,
            "kl_divergence"      : rec["label_kl_divergence"],
            "js_divergence"      : rec["label_js_divergence"],
            "mean_feature_js"    : rec["mean_feature_js"],
            "accuracy"           : acc_lookup[batch],
        })

    logger.info(f"Meta-dataset built: {len(rows)} samples")
    for r in rows:
        logger.info(
            f"  {r['batch'][:40]:<40} | "
            f"KL={r['kl_divergence']:.4f} | "
            f"JS={r['js_divergence']:.4f} | "
            f"Acc={r['accuracy']:.4f}"
        )
    return rows
```

```
def train_drift_predictor(rows: list):
    """
    Train regression models to predict accuracy from drift scores.
    With only 7 samples, we use leave-one-out cross validation
    to get an honest estimate of predictive performance.
    """
    X = np.array([[r["kl_divergence"], r["js_divergence"], r["mean_feature_js"]] for r in rows])
    y = np.array([r["accuracy"] for r in rows])

    feature_names = ["kl_divergence", "js_divergence", "mean_feature_js"]

    models = {
        "linear_regression"       : LinearRegression(),
        "gradient_boosting"       : GradientBoostingRegressor(
                                        n_estimators=50,
                                        max_depth=2,
                                        random_state=42
                                    ),
    }

    results = {}

    for name, model in models.items():
        # Leave-one-out cross validation (necessary with 7 samples)
        loo_preds = []
        loo_true  = []

        for i in range(len(X)):
            X_train = np.delete(X, i, axis=0)
            y_train = np.delete(y, i)
            X_test  = X[i:i+1]
            y_test  = y[i]

            model_copy = type(model)(**model.get_params())
            model_copy.fit(X_train, y_train)
            pred = model_copy.predict(X_test)[0]
            loo_preds.append(pred)
            loo_true.append(y_test)

        mae = mean_absolute_error(loo_true, loo_preds)
```

```
        r2  = r2_score(loo_true, loo_preds)

        logger.info(f"\n[{name}] LOO-CV MAE={mae:.4f} | R2={r2:.4f}")
        for actual, predicted in zip(loo_true, loo_preds):
            logger.info(f"  actual={actual:.4f}  predicted={predicted:.4f}  error={abs(actual-predicted):.4f}")

        # Train final model on all data
        model.fit(X, y)
        results[name] = {"model": model, "mae": mae, "r2": r2}

        # Save model
        os.makedirs(MODELS_PATH, exist_ok=True)
        model_path = os.path.join(MODELS_PATH, f"drift_predictor_{name}.pkl")
        joblib.dump(model, model_path)
        logger.info(f"Saved -> {model_path}")

        # Save report
        save_experiment_report({
            "Date"          : datetime.now().strftime("%Y-%m-%d %H:%M"),
            "Model"         : f"drift_predictor_{name}",
            "Task"          : "Drift prediction (accuracy regression)",
            "Input features": str(feature_names),
            "Samples"       : len(rows),
            "CV method"     : "Leave-one-out",
            "LOO-CV MAE"    : round(mae, 4),
            "LOO-CV R2"     : round(r2, 4),
        }, filename=f"drift_predictor_{name}.txt")

    return results

def predict_drift_risk(kl: float, js: float, mean_fjs: float,
                       model_name: str = "linear_regression") -> float:
    """
    Given new divergence scores, predict the expected model accuracy.
    Load the saved predictor and return a risk score.
    """
    model_path = os.path.join(MODELS_PATH, f"drift_predictor_{model_name}.pkl")
    model = joblib.load(model_path)
    X_new = np.array([[kl, js, mean_fjs]])
    predicted_accuracy = float(model.predict(X_new)[0])
    predicted_accuracy = max(0.0, min(1.0, predicted_accuracy))
```

```
    return predicted_accuracy

if __name__ == "__main__":
    logger.info("=== Drift Predictor Training Started ===")

    rows = load_meta_dataset()

    if len(rows) < 3:
        logger.error("Not enough samples to train. Run testdrift.py first.")
    else:
        results = train_drift_predictor(rows)

        logger.info("\n=== Final Comparison ===")
        for name, res in results.items():
            logger.info(f"{name:<30} MAE={res['mae']:.4f}  R2={res['r2']:.4f}")

        logger.info("\n=== Example Prediction ===")
        # Simulate a new batch with high drift
        acc_pred = predict_drift_risk(kl=0.8, js=0.25, mean_fjs=0.02)
        logger.info(f"For KL=0.8, JS=0.25, mean_fjs=0.02 -> predicted accuracy = {acc_pred:.4f}")

        acc_pred_low = predict_drift_risk(kl=0.01, js=0.004, mean_fjs=0.001)
        logger.info(f"For KL=0.01, JS=0.004, mean_fjs=0.001 -> predicted accuracy = {acc_pred_low:.4f}")

        logger.info("=== Drift Predictor Complete ===")

"""
drift.py
--------
All drift measurement logic for the project.

Contains:
  - Batch splitting (fixed reference and rolling window)
  - KL Divergence, Jensen-Shannon Divergence, Wasserstein Distance
  - Per-feature drift computation
  - Multi-model cross-dataset evaluation
  - Drift alert system
  - JSON output helpers
"""

import os
```

```
import json
import numpy as np
import pandas as pd
from scipy.stats import wasserstein_distance as scipy_wasserstein
from config import (
    REPORTS_PATH, TARGET_COLUMN,
    COMPUTE_KL, COMPUTE_JS, COMPUTE_WASSERSTEIN,
    DRIFT_ALERT_THRESHOLD, RANDOM_STATE,
)
from utils import get_logger

logger = get_logger("drift")

# ── Batch Splitting ──────────────────────────────────────────────────────────────────────────────

def split_by_source(df: pd.DataFrame) -> list:
    """
    Split into ordered batches by source file.
    Each batch = one day = one production time window.
    Returns list of (filename, dataframe) tuples.
    """
    from config import RAW_FILES
    batches = []
    for filename in RAW_FILES:
        group = df[df["source_file"] == filename]
        if len(group) == 0:
            logger.warning(f"No rows found for: {filename}")
            continue
        logger.info(f"Batch [{filename}]: {len(group)} rows")
        batches.append((filename, group))
    return batches

# ── Distribution Analysis ────────────────────────────────────────────────────────────────────────

def check_distribution(batches: list):
    """Print label distribution for each batch."""
    for name, batch in batches:
        dist = batch[TARGET_COLUMN].value_counts(normalize=True).sort_index()
        logger.info(f"[{name}] Label distribution:\n{dist}\n")

# ── Divergence Metrics ───────────────────────────────────────────────────────────────────────────
```

```
def kl_divergence(p: np.ndarray, q: np.ndarray) -> float:
    """KL Divergence D_KL(P || Q). Asymmetric."""
    p = np.array(p, dtype=float) + 1e-10
    q = np.array(q, dtype=float) + 1e-10
    p /= p.sum()
    q /= q.sum()
    return float(np.sum(p * np.log(p / q)))

def js_divergence(p: np.ndarray, q: np.ndarray) -> float:
    """Jensen-Shannon Divergence. Symmetric, bounded [0, 1]."""
    p = np.array(p, dtype=float) + 1e-10
    q = np.array(q, dtype=float) + 1e-10
    p /= p.sum()
    q /= q.sum()
    m = 0.5 * (p + q)
    return float(0.5 * np.sum(p * np.log(p / m)) + 0.5 * np.sum(q * np.log(q / m)))

def wasserstein_dist(a: np.ndarray, b: np.ndarray) -> float:
    """
    Wasserstein Distance (Earth Mover's Distance) between two 1D arrays.
    No binning required — operates directly on sample values.
    Larger = more distributional shift.
    """
    a = np.array(a, dtype=float)
    b = np.array(b, dtype=float)
    if len(a) == 0 or len(b) == 0:
        return 0.0
    return float(scipy_wasserstein(a, b))

# ── Per-Feature Drift ──────────────────────────────────────────────────────────────────────────

def compute_feature_drift(batch_a: pd.DataFrame,
                          batch_b: pd.DataFrame) -> dict:
    """
    Compute per-feature JS divergence between two batches.
    Shared bin edges ensure fair comparison across batches.
    Returns {feature_name: js_score}.
    """
    feature_cols = [
        c for c in batch_a.columns
```

```
        if c not in [TARGET_COLUMN, "source_file"]
        and pd.api.types.is_numeric_dtype(batch_a[c])
    ]
    drift_scores = {}
    for col in feature_cols:
        combined = pd.concat([batch_a[col], batch_b[col]])
        bins = np.histogram_bin_edges(combined.dropna(), bins=20)
        p, _ = np.histogram(batch_a[col].dropna(), bins=bins, density=True)
        q, _ = np.histogram(batch_b[col].dropna(), bins=bins, density=True)
        drift_scores[col] = round(js_divergence(p, q), 6)
    return drift_scores

def _label_metrics(ref_dist: pd.Series, curr_dist: pd.Series) -> dict:
    """Compute all configured label-level divergence metrics."""
    all_labels = sorted(set(ref_dist.index).union(curr_dist.index))
    ref_vals   = [ref_dist.get(k, 0) for k in all_labels]
    curr_vals  = [curr_dist.get(k, 0) for k in all_labels]
    metrics = {}
    if COMPUTE_KL:
        metrics["label_kl_divergence"] = round(kl_divergence(curr_vals, ref_vals), 6)
    if COMPUTE_JS:
        metrics["label_js_divergence"] = round(js_divergence(curr_vals, ref_vals), 6)
    if COMPUTE_WASSERSTEIN:
        metrics["label_wasserstein"]   = round(wasserstein_dist(curr_vals, ref_vals), 6)
    return metrics

def _feature_wasserstein_mean(ref_batch: pd.DataFrame,
                              curr_batch: pd.DataFrame) -> float:
    """Mean Wasserstein distance across all numeric features."""
    feat_cols = [
        c for c in ref_batch.columns
        if c not in [TARGET_COLUMN, "source_file"]
        and pd.api.types.is_numeric_dtype(ref_batch[c])
    ]
    scores = [
        wasserstein_dist(ref_batch[c].dropna().values,
                         curr_batch[c].dropna().values)
        for c in feat_cols
    ]
    return round(float(np.mean(scores)), 6)
```

```
# ── Fixed Reference Drift (Monday vs all) ──────────────────────────────────────

def compute_drift(batches: list) -> list:
    """
    Compare each batch against the fixed reference (Batch 0 = Monday).

    Computes per-comparison:
      - Label-level KL, JS, Wasserstein (as configured)
      - Feature-level mean JS and mean Wasserstein
      - Top 5 most drifted features

    Returns list of result dicts.
    """
    if len(batches) < 2:
        logger.warning("Need at least 2 batches to compute drift.")
        return []

    reference_name, reference_batch = batches[0]
    ref_dist = reference_batch[TARGET_COLUMN].value_counts(
        normalize=True).sort_index()

    results = []

    for i in range(1, len(batches)):
        curr_name, curr_batch = batches[i]
        curr_dist = curr_batch[TARGET_COLUMN].value_counts(
            normalize=True).sort_index()

        label_m     = _label_metrics(ref_dist, curr_dist)
        feature_js  = compute_feature_drift(reference_batch, curr_batch)
        mean_fjs    = round(float(np.mean(list(feature_js.values()))), 6)
        top_drifted = sorted(feature_js, key=feature_js.get, reverse=True)[:5]
        mean_ws     = _feature_wasserstein_mean(reference_batch, curr_batch) \
                      if COMPUTE_WASSERSTEIN else None

        result = {
            "reference_batch"          : reference_name,
            "comparison_batch"         : curr_name,
            "mean_feature_js"          : mean_fjs,
            "mean_feature_wasserstein" : mean_ws,
            "top_drifted_features"     : {f: feature_js[f] for f in top_drifted},
```

```
            **label_m,
        }
        results.append(result)

        logger.info(
            f"[Fixed] 0 vs {i} | "
            f"KL={label_m.get('label_kl_divergence', 0):.4f} | "
            f"JS={label_m.get('label_js_divergence', 0):.4f} | "
            f"WS={mean_ws:.4f} | FeatJS={mean_fjs:.4f} | "
            f"{curr_name}"
        )

    return results

# ── Rolling Reference Window ──────────────────────────────────────────────────────────────────

def compute_rolling_drift(batches: list) -> list:
    """
    Rolling reference window: compare batch[i] against batch[i-1].

    This simulates a realistic production monitor that checks whether
    the latest data window has shifted relative to the previous window,
    rather than always comparing to the original training distribution.

    Key difference from compute_drift():
      - compute_drift()       : Monday vs Tue, Mon vs Wed, Mon vs Thu ...
      - compute_rolling_drift(): Mon vs Tue, Tue vs Wed, Wed vs Thu ...

    Returns list of result dicts — one per consecutive pair.
    """
    if len(batches) < 2:
        logger.warning("Need at least 2 batches for rolling drift.")
        return []

    logger.info("\n--- Rolling Reference Window Drift ---")
    results = []

    for i in range(1, len(batches)):
        ref_name,  ref_batch  = batches[i - 1]
        curr_name, curr_batch = batches[i]
```

```
        ref_dist  = ref_batch[TARGET_COLUMN].value_counts(
            normalize=True).sort_index()
        curr_dist = curr_batch[TARGET_COLUMN].value_counts(
            normalize=True).sort_index()

        label_m     = _label_metrics(ref_dist, curr_dist)
        feature_js  = compute_feature_drift(ref_batch, curr_batch)
        mean_fjs    = round(float(np.mean(list(feature_js.values()))), 6)
        top_drifted = sorted(feature_js, key=feature_js.get, reverse=True)[:5]
        mean_ws     = _feature_wasserstein_mean(ref_batch, curr_batch) \
                      if COMPUTE_WASSERSTEIN else None

        result = {
            "reference_batch"          : ref_name,
            "comparison_batch"         : curr_name,
            "window_type"              : "rolling",
            "mean_feature_js"          : mean_fjs,
            "mean_feature_wasserstein" : mean_ws,
            "top_drifted_features"     : {f: feature_js[f] for f in top_drifted},
            **label_m,
        }
        results.append(result)

        logger.info(
            f"[Rolling] {i-1} -> {i} | "
            f"KL={label_m.get('label_kl_divergence', 0):.4f} | "
            f"JS={label_m.get('label_js_divergence', 0):.4f} | "
            f"WS={mean_ws:.4f} | "
            f"{ref_name} -> {curr_name}"
        )

    return results

# ── Save Results ──────────────────────────────────────────────────────────────

def save_drift_results(results: list,
                       filename: str = "drift_scores.json"):
    os.makedirs(REPORTS_PATH, exist_ok=True)
    path = os.path.join(REPORTS_PATH, filename)
    with open(path, "w") as f:
        json.dump(results, f, indent=4)
```

```
    logger.info(f"Drift results saved -> {path}")

def save_rolling_drift_results(results: list,
                               filename: str = "rolling_drift_scores.json"):
    os.makedirs(REPORTS_PATH, exist_ok=True)
    path = os.path.join(REPORTS_PATH, filename)
    with open(path, "w") as f:
        json.dump(results, f, indent=4)
    logger.info(f"Rolling drift results saved -> {path}")

def save_cross_eval_results(results: list,
                            filename: str = "cross_eval_accuracy.json"):
    os.makedirs(REPORTS_PATH, exist_ok=True)
    path = os.path.join(REPORTS_PATH, filename)
    with open(path, "w") as f:
        json.dump(results, f, indent=4)
    logger.info(f"Cross-eval results saved -> {path}")

# ── Multi-Model Cross-Dataset Evaluation ──────────────────────────────────────

def cross_dataset_evaluation(batches: list,
                             model_types: list = None) -> list:
    """
    CORE RESEARCH EXPERIMENT — multi-model version.

    For each model in model_types:
      1. Train on Batch 0 (Monday reference)
      2. Evaluate on every other batch without retraining
      3. Record accuracy per batch

    Scaling is applied internally using the reference batch only,
    to avoid data leakage from test batches into the scaler.
    SVM and Logistic Regression use scaled features.
    Random Forest, XGBoost, Decision Tree do not.

    Returns list of dicts: {model, batch, batch_idx, accuracy, note}
    """
    from sklearn.metrics import accuracy_score
    from sklearn.preprocessing import StandardScaler
    from models import get_model
```

```
if model_types is None:
    from config import CROSS_EVAL_MODELS
    model_types = CROSS_EVAL_MODELS

if len(batches) < 2:
    logger.warning("Need at least 2 batches for cross-dataset evaluation.")
    return []

reference_name, reference_batch = batches[0]
X_ref = reference_batch.drop(
    columns=[TARGET_COLUMN, "source_file"], errors="ignore")
y_ref = reference_batch[TARGET_COLUMN]

# Scaler fitted ONLY on reference batch
scaler = StandardScaler()
X_ref_scaled = scaler.fit_transform(X_ref)

all_results = []
NEEDS_SCALING = {"svm", "logistic"}

# ── Single-class guard ──────────────────────────────────────────────────────
# The Monday (reference) batch may be 100 % class 0 when DEBUG=True
# loads only a 10 % sample.  LogisticRegression and LinearSVC require
# at least 2 classes in y_train and raise ValueError if given only one.
# In that case we fall back to a DummyClassifier that always predicts the
# majority class — which is exactly what those models would do anyway —
# and record a note so the behaviour is transparent in the output.
n_ref_classes = y_ref.nunique()
if n_ref_classes < 2:
    logger.warning(
        f"Reference batch '{reference_name}' contains only "
        f"{n_ref_classes} class(es) — solvers that require 2+ classes "
        f"(logistic, svm) will use a DummyClassifier fallback."
    )

from sklearn.dummy import DummyClassifier

def _fit_model(model_type: str, model, X_train_arr, y_train):
    """
    Fit model or fall back to DummyClassifier when the training
    labels contain fewer classes than the solver requires.
```

```
    Returns (fitted_model, was_fallback: bool).
    """
    MULTI_CLASS_REQUIRED = {"logistic", "svm"}
    if model_type in MULTI_CLASS_REQUIRED and y_train.nunique() < 2:
        dummy = DummyClassifier(strategy="most_frequent",
                                random_state=RANDOM_STATE)
        dummy.fit(X_train_arr, y_train)
        return dummy, True
    model.fit(X_train_arr, y_train)
    return model, False

for model_type in model_types:
    logger.info(f"\n--- Cross-eval model: {model_type} ---")
    model = get_model(model_type)

    X_train = X_ref_scaled if model_type in NEEDS_SCALING else X_ref.values
    fitted_model, used_fallback = _fit_model(model_type, model, X_train, y_ref)

    if used_fallback:
        logger.warning(
            f"  [{model_type}] Reference is single-class — "
            f"using DummyClassifier (always predicts majority class)."
        )

    # Self-accuracy (upper bound)
    self_acc = accuracy_score(y_ref, fitted_model.predict(X_train))
    self_note = (
        "same distribution (upper bound) [DummyClassifier — single-class ref]"
        if used_fallback else
        "same distribution (upper bound)"
    )
    all_results.append({
        "model"    : model_type,
        "batch"    : reference_name,
        "batch_idx": 0,
        "accuracy" : round(self_acc, 4),
        "note"     : self_note,
    })
    logger.info(f"  Self-accuracy: {self_acc:.4f}")
```

```
        for i in range(1, len(batches)):
            curr_name, curr_batch = batches[i]
            X_test_raw = curr_batch.drop(
                columns=[TARGET_COLUMN, "source_file"], errors="ignore")
            y_test = curr_batch[TARGET_COLUMN]

            shared = [c for c in X_ref.columns if c in X_test_raw.columns]
            X_test_raw = X_test_raw[shared]

            X_test = scaler.transform(X_test_raw) \
                    if model_type in NEEDS_SCALING else X_test_raw.values

            acc = accuracy_score(y_test, fitted_model.predict(X_test))
            all_results.append({
                "model"    : model_type,
                "batch"    : curr_name,
                "batch_idx": i,
                "accuracy" : round(acc, 4),
                "note"     : (
                    "cross-dataset (drift scenario) [DummyClassifier]"
                    if used_fallback else
                    "cross-dataset (drift scenario)"
                ),
            })
            logger.info(f"  Batch {i} [{curr_name[:35]}]: {acc:.4f}")

    return all_results

# ── Drift Alert System ──────────────────────────────────────────────────────────────────────────

def check_drift_alerts(cross_eval_results: list,
                       drift_scores: list) -> list:
    """
    Check each batch's average cross-eval accuracy against
    DRIFT_ALERT_THRESHOLD. Writes drift_alerts.json if any
    batches fall below the threshold.

    Also calls the saved drift predictor if available.
    Returns list of alert dicts.
    """
    import joblib
```

```
from config import MODELS_PATH

drift_lookup = {r["comparison_batch"]: r for r in drift_scores}

# Average accuracy per batch across all models
batch_accs: dict = {}
batch_per_model: dict = {}
for r in cross_eval_results:
    b = r["batch"]
    if b not in batch_accs:
        batch_accs[b]     = []
        batch_per_model[b] = {}
    batch_accs[b].append(r["accuracy"])
    batch_per_model[b][r["model"]] = r["accuracy"]

predictor = None
predictor_path = os.path.join(MODELS_PATH,
                              "drift_predictor_linear_regression.pkl")
if os.path.exists(predictor_path):
    try:
        predictor = joblib.load(predictor_path)
    except Exception as e:
        logger.warning(f"Could not load drift predictor: {e}")

alerts = []
for batch, accs in batch_accs.items():
    avg_acc = float(np.mean(accs))
    if avg_acc >= DRIFT_ALERT_THRESHOLD:
        continue

    drift_info   = drift_lookup.get(batch, {})
    kl           = drift_info.get("label_kl_divergence")
    js           = drift_info.get("label_js_divergence")
    mfjs         = drift_info.get("mean_feature_js")
    predicted    = None

    if predictor and kl is not None and js is not None and mfjs is not None:
        try:
            predicted = round(float(
                np.clip(predictor.predict([[kl, js, mfjs]])[0], 0, 1)
            ), 4)
```

```
            except Exception:
                pass

        alert = {
            "batch"              : batch,
            "average_accuracy"   : round(avg_acc, 4),
            "threshold"          : DRIFT_ALERT_THRESHOLD,
            "per_model_accuracy" : batch_per_model[batch],
            "label_kl_divergence": kl,
            "label_js_divergence": js,
            "predicted_accuracy" : predicted,
            "alert"              : "DRIFT DETECTED — avg accuracy below threshold",
        }
        alerts.append(alert)
        logger.warning(
            f"DRIFT ALERT | {batch[:40]} | "
            f"avg_acc={avg_acc:.4f} < {DRIFT_ALERT_THRESHOLD}"
        )

    if alerts:
        path = os.path.join(REPORTS_PATH, "drift_alerts.json")
        os.makedirs(REPORTS_PATH, exist_ok=True)
        with open(path, "w") as f:
            json.dump(alerts, f, indent=4)
        logger.info(f"Alerts saved -> {path}")
    else:
        logger.info("No drift alerts triggered.")

    return alerts

"""
feature_engineering.py
-----------------------
Feature selection for the CICIDS 2017 pipeline.

Two-phase API (fit then transform)
===================================
The original single-pass apply_feature_engineering(df) required the full
2.8M-row combined dataset in memory so that variance and correlation could
be computed globally. On machines with limited RAM this caused an OOM crash
during the combined CSV read-back.
```

The new API separates the concerns:

```
  fit_feature_selector(df)
      Computes which columns to drop based on low variance and high
      correlation. Called ONCE on the Monday reference batch (~530k rows),
      which comfortably fits in memory. Returns a plain dict of column
      names to keep.

  apply_feature_selector(df, cols_to_keep)
      Drops every column not in cols_to_keep. Called on each day's
      interim file individually so only one file is in memory at a time.

  apply_feature_engineering(df)
      Original single-pass function retained for backward compatibility
      with any code that calls it directly (e.g. notebooks). It internally
      calls fit_feature_selector + apply_feature_selector, so it still
      works correctly when the full dataset is available.

Using Monday as the reference for feature selection is scientifically
sound: feature selection should be based on the training/reference
distribution only, not on test-day data, to avoid any form of leakage.
"""

import pandas as pd
import numpy as np
from sklearn.feature_selection import VarianceThreshold
from config import LOW_VARIANCE_THRESHOLD, CORRELATION_THRESHOLD, TARGET_COLUMN
from utils import get_logger

logger = get_logger("feature_engineering")

# ── Fit phase ──────────────────────────────────────────────────────────────────

def fit_feature_selector(df: pd.DataFrame) -> list:
    """
    Compute the list of numeric columns to retain after variance and
    correlation filtering.

    This function is intended to be called on the Monday reference batch
    only. The returned column list is then passed to apply_feature_selector()
```

```
for every subsequent day's data, ensuring no test-day information
influences which features are selected.

Parameters
----------
df : DataFrame
    Reference batch (Monday), excluding source_file. The Label /
    TARGET_COLUMN may be present; it is excluded from selection.

Returns
-------
list of str
    Column names to keep (numeric features that passed both filters,
    plus any non-numeric columns that were present).
"""
# Work only on numeric columns for the filters.
# Non-numeric columns (Label, source_file if present) are preserved as-is.
numeric_df = df.select_dtypes(include=[np.number])

# ── Step 1: variance filter ──────────────────────────────────────────────
selector = VarianceThreshold(threshold=LOW_VARIANCE_THRESHOLD)
selector.fit(numeric_df)
after_variance = numeric_df.columns[selector.get_support()].tolist()
n_removed_var  = len(numeric_df.columns) - len(after_variance)
logger.info(f"Low variance removed: {n_removed_var} features")

# ── Step 2: correlation filter ───────────────────────────────────────────
corr_df     = numeric_df[after_variance]
corr_matrix = corr_df.corr().abs()
upper       = corr_matrix.where(
    np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)
)
to_drop = [col for col in upper.columns if any(upper[col] > CORRELATION_THRESHOLD)]
after_corr = [c for c in after_variance if c not in to_drop]
logger.info(f"Highly correlated dropped: {len(to_drop)} features")
logger.info(f"Features retained after selection: {len(after_corr)}")

# Include non-numeric columns (e.g. Label) in the keep list so they
# are not accidentally dropped when apply_feature_selector() runs.
non_numeric = [c for c in df.columns if c not in numeric_df.columns]
cols_to_keep = after_corr + non_numeric
```

```
    return cols_to_keep

# ── Transform phase ──────────────────────────────────────────────────────────────────────────────

def apply_feature_selector(df: pd.DataFrame, cols_to_keep: list) -> pd.DataFrame:
    """
    Drop every column not present in cols_to_keep.

    Handles the case where a day's file may be missing some columns or
    have extra columns relative to the reference batch by intersecting
    rather than requiring an exact match.

    Parameters
    ----------
    df         : DataFrame for one day.
    cols_to_keep : Column list returned by fit_feature_selector().

    Returns
    -------
    Filtered DataFrame with only the selected columns (in original order).
    """
    present = [c for c in cols_to_keep if c in df.columns]
    dropped = len(df.columns) - len(present)
    if dropped:
        logger.info(f"  apply_feature_selector: dropped {dropped} columns")
    return df[present]

# ── Combined single-pass (backward-compatible) ──────────────────────────────────────────────────

def fit_feature_selector(df: pd.DataFrame) -> list:
    """
    Compute the list of numeric columns to retain after variance and
    correlation filtering.
    """
    # Work only on numeric columns for the filters.
    numeric_df = df.select_dtypes(include=[np.number])

    # ── FIX: Protect the Target Column from the Variance Filter ──
    if TARGET_COLUMN in numeric_df.columns:
        numeric_df = numeric_df.drop(columns=[TARGET_COLUMN])
```

```
    # ── Step 1: variance filter ──────────────────────────────────────────────
    selector = VarianceThreshold(threshold=LOW_VARIANCE_THRESHOLD)
    selector.fit(numeric_df)
    after_variance = numeric_df.columns[selector.get_support()].tolist()
    n_removed_var  = len(numeric_df.columns) - len(after_variance)
    logger.info(f"Low variance removed: {n_removed_var} features")

    # ── Step 2: correlation filter ───────────────────────────────────────────
    corr_df     = numeric_df[after_variance]
    corr_matrix = corr_df.corr().abs()
    upper       = corr_matrix.where(
        np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)
    )
    to_drop = [col for col in upper.columns if any(upper[col] > CORRELATION_THRESHOLD)]
    after_corr = [c for c in after_variance if c not in to_drop]
    logger.info(f"Highly correlated dropped: {len(to_drop)} features")
    logger.info(f"Features retained after selection: {len(after_corr)}")

    # Include non-numeric columns (e.g. Label) in the keep list so they
    # are not accidentally dropped when apply_feature_selector() runs.
    non_numeric = [c for c in df.columns if c not in numeric_df.columns]
    cols_to_keep = after_corr + non_numeric

    return cols_to_keep

from train import train_pipeline

if __name__ == "__main__":
    train_pipeline()
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.tree import DecisionTreeClassifier
from xgboost import XGBClassifier
from config import RANDOM_STATE

def get_model(model_type: str):
    """
    Return the requested model with sensible defaults.
```

```
Available models:
  random_forest  - ensemble bagging, strongest on tabular data
  xgboost        - gradient boosted trees, fast and accurate
  logistic       - linear baseline, fast, interpretable
  decision_tree  - single tree, fully interpretable, drift-sensitive
  svm            - LinearSVC, large-margin linear classifier

Note on SVM: Full kernel SVC is O(n^2) in memory and is unusable on
2.8M records. LinearSVC scales linearly and gives equivalent results
for linearly separable data. It does not expose predict_proba but
predict() works normally for accuracy evaluation and cross-eval.
"""
models = {
    "random_forest": RandomForestClassifier(
        n_estimators=100,
        random_state=RANDOM_STATE,
        n_jobs=-1,
        verbose=1,
    ),
    "xgboost": XGBClassifier(
        n_estimators=100,
        max_depth=6,
        learning_rate=0.1,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=RANDOM_STATE,
        n_jobs=-1,
        verbosity=1,
    ),
    "logistic": LogisticRegression(
        max_iter=1000,
        random_state=RANDOM_STATE,
        n_jobs=-1,
    ),
    "decision_tree": DecisionTreeClassifier(
        max_depth=10,
        min_samples_leaf=50,
        random_state=RANDOM_STATE,
    ),
    "svm": LinearSVC(
        C=1.0,
```

```
            max_iter=2000,
            random_state=RANDOM_STATE,
        ),
    }

    if model_type not in models:
        raise ValueError(
            f"Unknown model type: '{model_type}'. "
            f"Choose from: {list(models.keys())}"
        )

    return models[model_type]

def get_shap_explainer_type(model_type: str) -> str:
    """
    Return which SHAP explainer backend to use for a given model type.
    Called by shap_analysis.py to select the correct explainer.
    """
    if model_type in {"random_forest", "xgboost", "decision_tree"}:
        return "tree"
    elif model_type in {"logistic", "svm"}:
        return "linear"
    return "unknown"

"""
preprocessing.py
----------------
Data loading and cleaning pipeline for CICIDS 2017.

Memory strategy — definitive, architecture-level fix
=====================================================
The fundamental constraint is that this machine cannot hold the full
2.8M-row combined dataset (roughly 1.8 GB as float64, ~1.1 GB as float32)
in memory in one contiguous allocation. Every approach that eventually calls
pd.concat() or pd.read_csv() on the full combined data will fail on this
hardware, regardless of how clean the chunks are.

The correct solution is to NEVER combine all 8 files into a single
in-memory DataFrame at any point during preprocessing:

  Stage 1 — Per-file streaming to disk
```

```
    Each raw CSV is read in 20,000-row chunks. Every chunk is cleaned
    (inf replacement via NumPy, NaN drop, float32 downcast) and written
    to an individual interim CSV at data/interim/<filename>.csv.
    Peak memory: one 20,000-row chunk (~6 MB).

  Stage 2 — Feature selector fitted on Monday only
    The Monday interim file (~530k rows, ~200 MB) is loaded in isolation.
    VarianceThreshold and correlation filtering are fitted on this file.
    The resulting list of columns to keep is recorded.
    Monday is used as the reference because feature selection should be
    based on the training/reference distribution only — using all 8 days
    would allow test-day information to influence feature selection, which
    is a form of data leakage.
    Peak memory: Monday interim file (~200 MB).

  Stage 3 — Per-file transform and append to processed CSV
    Each interim file is loaded one at a time, the pre-computed column
    list is applied, unnecessary columns are dropped, labels are encoded,
    and the result is appended to data/processed/processed_data_v1.csv.
    Peak memory: one interim file (~100–470 MB).

The final processed CSV is then read back one more time during training
(unavoidable — sklearn needs the full matrix), but that read is a single
fresh pd.read_csv() which Pandas handles without triggering block
consolidation because it builds column blocks from scratch.

If that final read still fails due to hardware limitations, set
DEBUG = True in config.py to work with a 100,000-row sample, or increase
your system's virtual memory / page file on Windows.
"""

import gc
import os

import numpy as np
import pandas as pd
import joblib
from sklearn.preprocessing import StandardScaler

from config import *
from utils import get_logger
```

```
from feature_engineering import fit_feature_selector, apply_feature_selector

logger = get_logger("preprocessing")

# ── Interim file path helper ──────────────────────────────────────────────────

def _interim_path(filename: str) -> str:
    """Return the interim CSV path for a given raw source filename."""
    return os.path.join(INTERIM_DATA_PATH, filename + ".interim.csv")

# ── Dtype optimisation ────────────────────────────────────────────────────────

def _downcast_floats(chunk: pd.DataFrame) -> pd.DataFrame:
    """
    Convert float64 columns to float32, one column at a time.
    Per-column assignment avoids triggering Pandas block consolidation,
    which requires a large contiguous allocation across all float columns.
    """
    for col in chunk.select_dtypes(include=["float64"]).columns:
        chunk[col] = chunk[col].astype(np.float32)
    return chunk

# ── Per-chunk cleaning ────────────────────────────────────────────────────────

def _clean_chunk(chunk: pd.DataFrame) -> pd.DataFrame:
    """
    Clean a single 20,000-row chunk:
      1. Strip leading/trailing whitespace from column names.
      2. Replace +/-inf with NaN using NumPy (avoids Pandas replace() path).
      3. Drop NaN rows.
      4. Downcast float64 to float32.
    """
    chunk.columns = chunk.columns.str.strip()

    float_cols = chunk.select_dtypes(include=["float32", "float64"]).columns
    if len(float_cols):
        arr = chunk[float_cols].to_numpy(dtype=np.float64, copy=True)
        arr[~np.isfinite(arr)] = np.nan
        chunk[float_cols] = arr

    chunk.dropna(inplace=True)
```

```
    chunk = _downcast_floats(chunk)
    return chunk

# ── Stage 1: stream each file to its own interim CSV ──────────────────────────

def _stream_file_to_interim(filename: str) -> int:
    """
    Read one raw CICIDS CSV in 20,000-row chunks, clean each chunk,
    and write it to data/interim/<filename>.interim.csv.

    Returns the number of rows successfully written.
    """
    filepath = os.path.join(RAW_DATA_PATH, filename)
    out_path = _interim_path(filename)

    if not os.path.exists(filepath):
        logger.warning(f"File not found, skipping: {filename}")
        return 0

    # Remove stale interim from a previous failed run.
    if os.path.exists(out_path):
        os.remove(out_path)

    rows_written = 0
    rows_removed = 0
    header_done  = False

    for chunk in pd.read_csv(filepath, chunksize=20_000, low_memory=False):
        chunk["source_file"] = filename
        before       = len(chunk)
        chunk        = _clean_chunk(chunk)
        rows_removed += before - len(chunk)

        if len(chunk) == 0:
            continue

        chunk.to_csv(out_path, mode="a", header=not header_done, index=False)
        header_done   = True
        rows_written += len(chunk)
        del chunk
        gc.collect()
```

```
    if rows_removed:
        logger.info(
            f"  [{filename[:30]}] "
            f"kept {rows_written:,} rows, removed {rows_removed:,} NaN/Inf rows"
        )
    else:
        logger.info(f"  [{filename[:30]}] kept {rows_written:,} rows")

    return rows_written

def stream_all_files_to_interim():
    """Run Stage 1 for every file in RAW_FILES."""
    os.makedirs(INTERIM_DATA_PATH, exist_ok=True)
    total = 0
    for filename in RAW_FILES:
        logger.info(f"Streaming: {filename}")
        total += _stream_file_to_interim(filename)
        gc.collect()
    logger.info(f"Stage 1 complete: {total:,} rows written to interim/")

# ── Stage 2: fit feature selector on Monday only ──────────────────────────────

def fit_selector_on_reference():
    reference_file = RAW_FILES[0]
    ref_path = _interim_path(reference_file)

    logger.info("Sampling reference data for feature selection")

    ref_df = None

    for chunk in pd.read_csv(ref_path, chunksize=20000, low_memory=False):
        chunk = chunk.sample(frac=0.2, random_state=42)

        cols_to_drop = [c for c in COLUMNS_TO_DROP if c in chunk.columns]
        chunk.drop(columns=cols_to_drop, inplace=True)

        if "source_file" in chunk.columns:
            chunk.drop(columns=["source_file"], inplace=True)

        chunk = _encode_labels(chunk)
```

```
        if ref_df is None:
            ref_df = chunk
        else:
            ref_df = pd.concat([ref_df, chunk], ignore_index=True)

        del chunk
        gc.collect()

    cols_to_keep = fit_feature_selector(ref_df)

    del ref_df
    gc.collect()

    return cols_to_keep

# ── Label encoding helper ──────────────────────────────────────────────────────────────────────

def _encode_labels(df: pd.DataFrame) -> pd.DataFrame:
    """BENIGN -> 0, all attack labels -> 1 (int8)."""
    if TARGET_COLUMN in df.columns:
        df[TARGET_COLUMN] = df[TARGET_COLUMN].str.strip()
        df[TARGET_COLUMN] = (df[TARGET_COLUMN] != "BENIGN").astype(np.int8)
    return df

# ── Stage 3: transform each interim file and append to processed CSV ──────────────

def build_processed_csv(cols_to_keep: list):
    processed_path = os.path.join(PROCESSED_DATA_PATH, PROCESSED_FILENAME)
    os.makedirs(PROCESSED_DATA_PATH, exist_ok=True)

    if os.path.exists(processed_path):
        os.remove(processed_path)

    header_done   = False
    total_written = 0

    for filename in RAW_FILES:
        interim_path = _interim_path(filename)

        if not os.path.exists(interim_path):
```

```
        logger.warning(f"Interim file missing, skipping: {filename}")
        continue

    logger.info(f"Processing: {filename}")

    for chunk in pd.read_csv(interim_path, chunksize=50000, low_memory=False):

        cols_drop = [c for c in COLUMNS_TO_DROP if c in chunk.columns]
        chunk.drop(columns=cols_drop, inplace=True)

        source_col = None
        if "source_file" in chunk.columns:
            source_col = chunk["source_file"].copy()
            chunk.drop(columns=["source_file"], inplace=True)

        chunk = _encode_labels(chunk)
        chunk = apply_feature_selector(chunk, cols_to_keep)

        if source_col is not None:
            chunk["source_file"] = source_col.values

        chunk.to_csv(
            processed_path,
            mode="a",
            header=not header_done,
            index=False
        )

        header_done = True
        total_written += len(chunk)

        logger.info(
            f"  Written {len(chunk):,} rows "
            f"(total so far: {total_written:,})"
        )

        del chunk, source_col
        gc.collect()

logger.info(f"Stage 3 complete: {total_written:,} rows in processed CSV")
```

```
# ── Constant column removal (across combined dataset) ──────────────────────────

# ── Constant column removal (corrected logic) ──────────────────────────

def _find_constant_columns() -> list:
    processed_path = os.path.join(PROCESSED_DATA_PATH, PROCESSED_FILENAME)

    unique_vals = {}

    for chunk in pd.read_csv(processed_path, chunksize=20000, low_memory=False):
        for col in chunk.columns:
            if col in ["source_file", TARGET_COLUMN]:
                continue

            vals = set(chunk[col].dropna().unique())

            if col not in unique_vals:
                unique_vals[col] = vals
            else:
                # Only track until >1 unique (optimization)
                if len(unique_vals[col]) <= 1:
                    unique_vals[col].update(vals)

        del chunk
        gc.collect()

    constant = [col for col, vals in unique_vals.items() if len(vals) <= 1]
    return constant

def _drop_constant_columns(constant_cols: list):
    """
    Remove constant columns from the processed CSV by rewriting it in
    chunks.  Overwrites the file in-place.
    """
    if not constant_cols:
        logger.info("No constant columns found across combined dataset.")
        return

    logger.info(f"Removing {len(constant_cols)} constant columns from processed CSV.")
    processed_path = os.path.join(PROCESSED_DATA_PATH, PROCESSED_FILENAME)
    tmp_path       = processed_path + ".tmp"
```

```
    header_done = False
    for chunk in pd.read_csv(processed_path, chunksize=20_000, low_memory=False):
        chunk.drop(columns=constant_cols, errors="ignore", inplace=True)
        chunk.to_csv(tmp_path, mode="a", header=not header_done, index=False)
        header_done = True
        del chunk
        gc.collect()

    os.replace(tmp_path, processed_path)
    logger.info("Constant column removal complete.")

# ── Scaling ──────────────────────────────────────────────────────────────────────────────────

def scale_features(X_train, X_test):
    """
    Fit StandardScaler on the training split only, then transform both.
    Fitted on training data exclusively to prevent test-set leakage.
    """
    scaler         = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled  = scaler.transform(X_test)

    if SAVE_SCALER:
        os.makedirs(SCALERS_PATH, exist_ok=True)
        path = os.path.join(SCALERS_PATH, "scaler_v1.pkl")
        joblib.dump(scaler, path)
        logger.info(f"Scaler saved -> {path}")

    return X_train_scaled, X_test_scaled, scaler

# ── Full pipeline ────────────────────────────────────────────────────────────────────────────

def run_preprocessing() -> pd.DataFrame:
    """
    Full preprocessing pipeline.

    Stage 1 — Streaming write
      Each raw file is cleaned chunk-by-chunk and written to its own
      interim CSV. No combined DataFrame is ever built.
```

```
Stage 2 — Feature selection fitted on Monday
  VarianceThreshold and correlation filter are fitted on the Monday
  interim file (~530k rows). The selected column list is saved.

Stage 3 — Per-file transform and final CSV construction
  Each interim file is loaded, the column selection is applied, and
  the result is appended to the processed CSV.

Stage 4 — Constant column scan (chunked)
  The processed CSV is scanned in 50,000-row chunks to find any
  globally constant columns, which are then removed.

Returns
-------
pd.DataFrame
    The fully processed dataset, loaded from the final CSV.
    This is the one unavoidable full-dataset load (needed for
    train_test_split and model training in train.py).
"""
# Stage 1
logger.info("=== Stage 1: Streaming raw files to interim/ ===")
stream_all_files_to_interim()

# Stage 2
logger.info("=== Stage 2: Fitting feature selector on Monday ===")
cols_to_keep = fit_selector_on_reference()

# Stage 3
logger.info("=== Stage 3: Building processed CSV ===")
build_processed_csv(cols_to_keep)

# Stage 4
logger.info("=== Stage 4: Scanning for globally constant columns ===")
constant_cols = _find_constant_columns()
_drop_constant_columns(constant_cols)

# ── Final load (OOM-Safe) ──────────────────────────────────────────────────
processed_path = os.path.join(PROCESSED_DATA_PATH, PROCESSED_FILENAME)
logger.info(f"Loading processed dataset: {processed_path}")

if DEBUG:
```

```
        logger.info("DEBUG mode: Chunk-loading a sample to prevent memory crash...")
        chunk_list = []
        # Read the file in small chunks so it never overwhelms your RAM
        for chunk in pd.read_csv(processed_path, chunksize=50_000, low_memory=False):
            # Keep only 5% of each chunk
            chunk_list.append(chunk.sample(frac=0.05, random_state=RANDOM_STATE))

        # Combine the tiny chunks into one safe DataFrame
        df = pd.concat(chunk_list, ignore_index=True)

        # Cap it exactly at 100,000 rows if it exceeded it
        if len(df) > 100_000:
            df = df.sample(100_000, random_state=RANDOM_STATE)
    else:
        # Warning: This requires ~2GB+ of free RAM
        df = pd.read_csv(processed_path, low_memory=False)

    mem_mb = df.memory_usage(deep=True).sum() / 1e6
    logger.info(f"Loaded {len(df):,} rows | Memory: {mem_mb:.1f} MB")

    if "source_file" not in df.columns:
        raise ValueError(
            "source_file column is missing from the processed dataset. "
            "Check that RAW_FILES in config.py matches actual filenames."
        )

    return df

"""
shap_analysis.py
----------------
SHAP feature importance analysis under data drift.

Answers: when a model is tested on a drifted batch, which features
are driving its predictions, and how do they differ from the reference?

Explainer selection:
  random_forest, xgboost, decision_tree -> TreeExplainer  (fast, exact)
  logistic, svm                         -> LinearExplainer (fast, exact)

Run from src/:
```

```
  pip install shap matplotlib
  python shap_analysis.py

Output (in artifacts/reports/):
  shap_comparison_{model}.png     side-by-side bar chart
  shap_values_{model}.json        mean |SHAP| per feature
"""

import os
import json
import numpy as np
import pandas as pd
import joblib
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

from config import (
    PROCESSED_DATA_PATH, PROCESSED_FILENAME,
    TARGET_COLUMN, MODELS_PATH, REPORTS_PATH,
    RANDOM_STATE, SHAP_SAMPLE_SIZE, SHAP_TOP_N_FEATURES, DEBUG,
)
from models import get_shap_explainer_type
from utils import get_logger

logger = get_logger("shap_analysis")

SHAP_MODELS         = ["random_forest", "xgboost", "decision_tree", "logistic", "svm"]
REFERENCE_BATCH_IDX = 0   # Monday
DRIFTED_BATCH_IDX   = 7   # Friday DDoS — highest drift

def load_processed_data() -> pd.DataFrame:
    path = os.path.join(PROCESSED_DATA_PATH, PROCESSED_FILENAME)
    if not os.path.exists(path):
        raise FileNotFoundError(f"Processed data not found: {path}")
    logger.info(f"Loading: {path}")

    if DEBUG:
        logger.info("DEBUG mode: Chunk-loading to prevent memory crash...")
        chunk_list = []
        for chunk in pd.read_csv(path, chunksize=50_000, low_memory=False):
```

```
            # Keep 10% of each chunk to maintain distribution without crashing RAM
            chunk_list.append(chunk.sample(frac=0.10, random_state=RANDOM_STATE))
        return pd.concat(chunk_list, ignore_index=True)
    else:
        return pd.read_csv(path, low_memory=False)

def get_batches(df: pd.DataFrame):
    from config import RAW_FILES
    ref_file     = RAW_FILES[REFERENCE_BATCH_IDX]
    drifted_file = RAW_FILES[DRIFTED_BATCH_IDX]
    ref     = df[df["source_file"] == ref_file]
    drifted = df[df["source_file"] == drifted_file]
    logger.info(f"Reference : {ref_file} ({len(ref)} rows)")
    logger.info(f"Drifted   : {drifted_file} ({len(drifted)} rows)")
    return ref, drifted, ref_file, drifted_file

def _prep(batch, feature_cols, needs_scaling, scaler):
    """Sample and optionally scale a batch for SHAP computation."""
    s = batch.sample(min(SHAP_SAMPLE_SIZE, len(batch)), random_state=RANDOM_STATE)
    X = s[feature_cols].values
    if needs_scaling and scaler is not None:
        X = scaler.transform(s[feature_cols])
    return X

def run_shap_for_model(model_type, ref_batch, drifted_batch, ref_name, drifted_name):
    try:
        import shap
    except ImportError:
        logger.error("SHAP not installed. Run: pip install shap")
        return

    model_path = os.path.join(MODELS_PATH, f"{model_type}_v1.pkl")
    if not os.path.exists(model_path):
        logger.warning(f"Model not found: {model_path} — train it first with main.py")
        return

    logger.info(f"\n--- SHAP: {model_type} ---")
    model  = joblib.load(model_path)

    scaler_path   = os.path.join(os.path.dirname(MODELS_PATH), "scalers", "scaler_v1.pkl")
    scaler        = joblib.load(scaler_path) if os.path.exists(scaler_path) else None
```

```
needs_scaling = model_type in {"svm", "logistic"}
exp_type      = get_shap_explainer_type(model_type)

feature_cols = [
    c for c in ref_batch.columns
    if c not in [TARGET_COLUMN, "source_file"]
    and pd.api.types.is_numeric_dtype(ref_batch[c])
]

X_ref     = _prep(ref_batch,     feature_cols, needs_scaling, scaler)
X_drifted = _prep(drifted_batch, feature_cols, needs_scaling, scaler)

try:
    if exp_type == "tree":
        explainer    = shap.TreeExplainer(model)
        sv_ref       = explainer.shap_values(X_ref)
        sv_drifted   = explainer.shap_values(X_drifted)

    elif exp_type == "linear":
        explainer    = shap.LinearExplainer(model, X_ref[:100])
        sv_ref       = explainer.shap_values(X_ref)
        sv_drifted   = explainer.shap_values(X_drifted)

    else:
        logger.warning(f"No SHAP explainer for {model_type}")
        return

    # ── FIX: Safely extract Class 1 SHAP values for any SHAP version ──
    if isinstance(sv_ref, list):
        # Older SHAP versions return a list
        sv_ref     = sv_ref[1]
        sv_drifted = sv_drifted[1]
    elif len(sv_ref.shape) == 3:
        # Newer SHAP versions return a 3D array (samples, features, classes)
        sv_ref     = sv_ref[:, :, 1]
        sv_drifted = sv_drifted[:, :, 1]

except Exception as e:
    logger.error(f"SHAP failed for {model_type}: {e}")
    return
```

```
mean_ref     = np.abs(sv_ref).mean(axis=0)
mean_drifted = np.abs(sv_drifted).mean(axis=0)

top_idx      = np.argsort(mean_ref)[-SHAP_TOP_N_FEATURES:][::-1]
top_features = [feature_cols[i] for i in top_idx]
ref_vals     = [round(float(mean_ref[i]),     6) for i in top_idx]
drift_vals   = [round(float(mean_drifted[i]), 6) for i in top_idx]

# ── Side-by-side plot ──────────────────────────────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(16, 7))
fig.suptitle(
    f"SHAP Feature Importance Under Drift  |  Model: {model_type}\n"
    f"Left: Reference (Monday)   Right: Drifted (Friday DDoS)",
    fontsize=12
)
for ax, vals, title, colour in [
    (axes[0], ref_vals,   f"Reference\n{ref_name[:45]}",     "#2E86AB"),
    (axes[1], drift_vals, f"Drifted\n{drifted_name[:45]}", "#E84855"),
]:
    y = range(len(top_features))
    ax.barh(list(y), vals, color=colour, alpha=0.85, edgecolor="none")
    ax.set_yticks(list(y))
    ax.set_yticklabels(top_features, fontsize=8)
    ax.set_xlabel("Mean |SHAP value|", fontsize=10)
    ax.set_title(title, fontsize=10)
    ax.invert_yaxis()
    ax.grid(axis="x", alpha=0.3)

plt.tight_layout()
os.makedirs(REPORTS_PATH, exist_ok=True)
plot_path = os.path.join(REPORTS_PATH, f"shap_comparison_{model_type}.png")
plt.savefig(plot_path, dpi=150, bbox_inches="tight")
plt.close()
logger.info(f"Plot saved -> {plot_path}")

# ── JSON summary ───────────────────────────────────────────────────────────
summary = {
    "model"          : model_type,
    "reference_batch": ref_name,
    "drifted_batch"  : drifted_name,
    "top_features"   : top_features,
```

```
        "reference_mean_abs_shap": dict(zip(top_features, ref_vals)),
        "drifted_mean_abs_shap"  : dict(zip(top_features, drift_vals)),
    }
    json_path = os.path.join(REPORTS_PATH, f"shap_values_{model_type}.json")
    with open(json_path, "w") as f:
        json.dump(summary, f, indent=4)
    logger.info(f"JSON saved -> {json_path}")

    # ── Log top 5 most shifted features ──────────────────────────────────────
    logger.info("  Top 5 features — reference vs drifted importance:")
    for feat, rv, dv in zip(top_features[:5], ref_vals[:5], drift_vals[:5]):
        change = dv - rv
        logger.info(
            f"    {feat:<38} ref={rv:.4f}  drift={dv:.4f}  "
            f"delta={change:+.4f}"
        )

def run_shap_analysis():
    logger.info("=== SHAP Analysis Started ===")
    df = load_processed_data()
    ref_batch, drifted_batch, ref_name, drifted_name = get_batches(df)

    for model_type in SHAP_MODELS:
        run_shap_for_model(model_type, ref_batch, drifted_batch,
                           ref_name, drifted_name)

    logger.info("\n=== SHAP Analysis Complete ===")
    logger.info(f"All outputs in: {REPORTS_PATH}")

if __name__ == "__main__":
    run_shap_analysis()

"""
testdrift.py
------------
Main drift experiment runner.

Runs in this order:
  1. Load processed data
  2. Split into 8 temporal batches
  3. Check label distributions
```

```
  4. Fixed-reference drift  (Monday vs all)  -> drift_scores.json
  5. Rolling-window drift   (consecutive)    -> rolling_drift_scores.json
  6. Multi-model cross-dataset evaluation    -> cross_eval_accuracy.json
  7. Drift alert check                       -> drift_alerts.json (if needed)
  8. Print summary table

Run from src/:
  python testdrift.py
"""

import os
import pandas as pd
from config import (
    PROCESSED_DATA_PATH, PROCESSED_FILENAME,
    TARGET_COLUMN, MODELS_PATH, REPORTS_PATH,
    RANDOM_STATE, SHAP_SAMPLE_SIZE, SHAP_TOP_N_FEATURES,
    DEBUG
)
from drift import (
    split_by_source,
    check_distribution,
    compute_drift,
    save_drift_results,
    compute_rolling_drift,
    save_rolling_drift_results,
    cross_dataset_evaluation,
    save_cross_eval_results,
    check_drift_alerts,
)
from utils import get_logger

logger = get_logger("testdrift")

def run_drift_experiment():
    logger.info("=== Drift Experiment Started ===")


    # ── 1. Load processed data ──────────────────────────────────────────────
    processed_path = os.path.join(PROCESSED_DATA_PATH, PROCESSED_FILENAME)
    if not os.path.exists(processed_path):
        logger.error(f"Processed data not found: {processed_path}")
```

```
    logger.error("Run main.py first.")
    return

logger.info(f"Loading: {processed_path}")

# NEW OOM-SAFE LOADING LOGIC
if DEBUG:
    logger.info("DEBUG mode: Chunk-loading to prevent memory crash...")
    chunk_list = []
    for chunk in pd.read_csv(processed_path, chunksize=50_000, low_memory=False):
        # Keep 10% of each chunk to maintain temporal distribution
        chunk_list.append(chunk.sample(frac=0.10, random_state=RANDOM_STATE))
    df = pd.concat(chunk_list, ignore_index=True)
else:
    df = pd.read_csv(processed_path, low_memory=False)

logger.info(f"Total rows loaded: {len(df)}")

# ── 2. Split into batches ──────────────────────────────────────────────────
batches = split_by_source(df)
logger.info(f"Total batches found: {len(batches)}")

if len(batches) < 2:
    logger.warning(
        "Only 1 batch found. Make sure all 8 CICIDS files are in "
        "RAW_FILES in config.py and preprocessing has been re-run."
    )
    return

# ── 3. Distribution check ──────────────────────────────────────────────────
logger.info("\n--- Label Distribution per Batch ---")
check_distribution(batches)

# ── 4. Fixed-reference drift (Monday vs all) ───────────────────────────────
logger.info("\n--- Fixed Reference Drift (Monday vs all batches) ---")
drift_results = compute_drift(batches)
save_drift_results(drift_results)

# ── 5. Rolling-window drift (consecutive batches) ──────────────────────────
if USE_ROLLING_REFERENCE:
    rolling_results = compute_rolling_drift(batches)
```

```
    save_rolling_drift_results(rolling_results)

# ── 6. Multi-model cross-dataset evaluation ──────────────────────────────
logger.info(
    f"\n--- Multi-Model Cross-Dataset Evaluation ---\n"
    f"Models: {CROSS_EVAL_MODELS}"
)
eval_results = cross_dataset_evaluation(batches, model_types=CROSS_EVAL_MODELS)
save_cross_eval_results(eval_results)

# ── 7. Drift alerts ──────────────────────────────────────────────────────
logger.info("\n--- Checking Drift Alerts ---")
alerts = check_drift_alerts(eval_results, drift_results)

# ── 8. Summary table ─────────────────────────────────────────────────────
logger.info("\n=== CROSS-DATASET EVALUATION SUMMARY ===")

# Collect unique batches in order
batch_order = []
seen = set()
for r in eval_results:
    if r["batch"] not in seen:
        batch_order.append(r["batch"])
        seen.add(r["batch"])

# Build per-batch per-model accuracy dict
acc_table: dict = {b: {} for b in batch_order}
for r in eval_results:
    acc_table[r["batch"]][r["model"]] = r["accuracy"]

models_seen = CROSS_EVAL_MODELS
col_w       = 12

header = f"{'Batch':<52}" + "".join(f"{m:>{col_w}}" for m in models_seen)
logger.info(header)
logger.info("-" * len(header))

for batch in batch_order:
    row = f"{batch:<52}"
    for m in models_seen:
        val = acc_table[batch].get(m, "N/A")
```

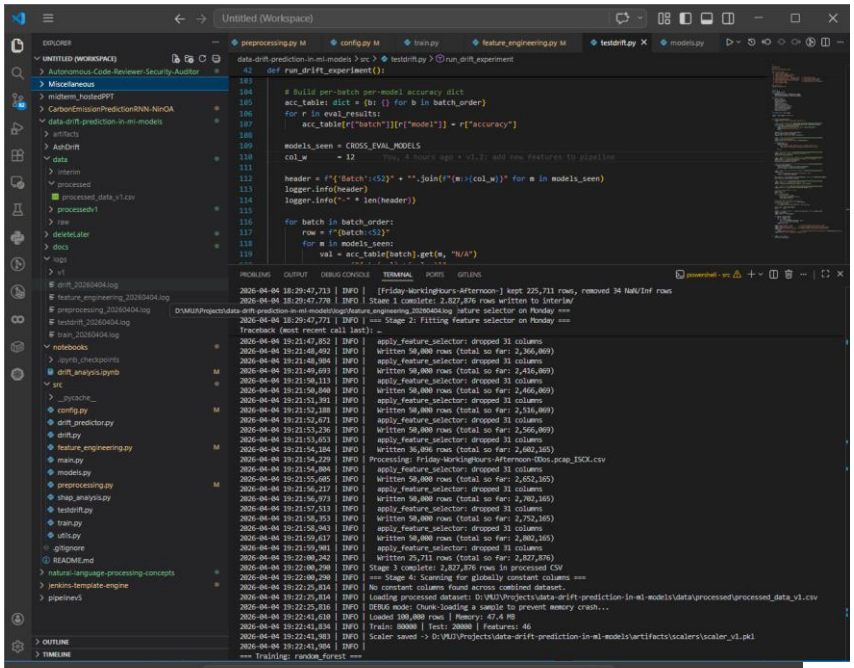
```
            row += f"{str(val):>{col_w}}"
        logger.info(row)

    logger.info(f"\nDrift scores          -> artifacts/reports/drift_scores.json")
    logger.info(f"Rolling drift        -> artifacts/reports/rolling_drift_scores.json")
    logger.info(f"Cross-eval accuracy  -> artifacts/reports/cross_eval_accuracy.json")
    if alerts:
        logger.info(f"Drift alerts         -> artifacts/reports/drift_alerts.json  ({len(alerts)} alerts)")
    logger.info("=== Drift Experiment Complete ===")

if __name__ == "__main__":
    run_drift_experiment()

"""
train.py
--------
Baseline model training pipeline.

Trains all models in MODELS_TO_RUN (config.py) on the full combined
processed dataset, evaluates on the held-out test split, saves each
model and its experiment report.

Decision Tree and SVM are now included alongside the original three.

Note on SVM training time:
  LinearSVC on 2.26M training samples takes approximately 5-15 minutes
  depending on hardware. It is placed last in MODELS_TO_RUN so the
  other models complete quickly before it runs.

Run from src/:
  python main.py
"""

import os
import joblib
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix
)
from datetime import datetime
```

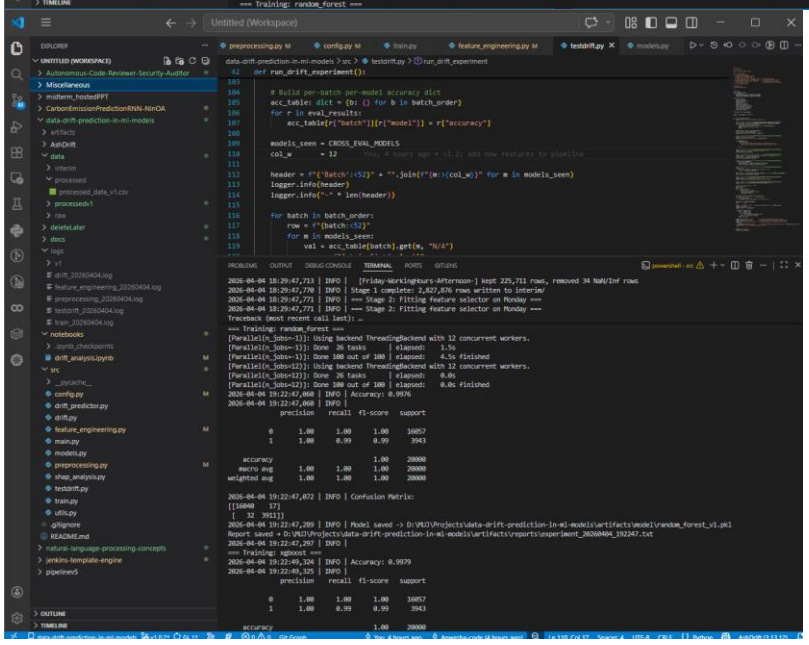
```
from config import *
from preprocessing import run_preprocessing, scale_features
from models import get_model
from utils import get_logger, save_experiment_report

logger = get_logger("train")

def train_pipeline():
    logger.info("=== Training Pipeline Started ===")

    # ── Load or generate processed data ──────────────────────────────────
    processed_path = os.path.join(PROCESSED_DATA_PATH, PROCESSED_FILENAME)

    if os.path.exists(processed_path):
        logger.info(f"Loading existing processed data: {processed_path}")
        df = pd.read_csv(processed_path)
    else:
        logger.info("Processed data not found. Running preprocessing...")
        df = run_preprocessing()

    # ── Split features / target ──────────────────────────────────────────
    X = df.drop(columns=[TARGET_COLUMN, "source_file"], errors="ignore")
    y = df[TARGET_COLUMN]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE, stratify=y
    )
    logger.info(f"Train: {len(X_train)} | Test: {len(X_test)} | Features: {X.shape[1]}")

    # ── Scale features ───────────────────────────────────────────────────
    # Scaler is always fitted so that SVM and Logistic get proper input.
    # Tree models receive the scaled arrays too — scaling does not affect
    # their predictions, but it keeps the pipeline uniform.
    X_train_sc, X_test_sc, _ = scale_features(X_train, X_test)

    # Models that require scaled input
    NEEDS_SCALING = {"svm", "logistic"}

    # ── Train each model ─────────────────────────────────────────────────
    for model_type in MODELS_TO_RUN:
```

```
logger.info(f"\n=== Training: {model_type} ===")

model   = get_model(model_type)
X_tr    = X_train_sc if model_type in NEEDS_SCALING else X_train.values
X_te    = X_test_sc  if model_type in NEEDS_SCALING else X_test.values

model.fit(X_tr, y_train)

preds   = model.predict(X_te)
acc     = accuracy_score(y_test, preds)
report  = classification_report(y_test, preds)
cm      = confusion_matrix(y_test, preds)

logger.info(f"Accuracy: {acc:.4f}")
logger.info(f"\n{report}")
logger.info(f"Confusion Matrix:\n{cm}")

# Save predictions CSV
os.makedirs(REPORTS_PATH, exist_ok=True)
pred_df = pd.DataFrame({"y_true": y_test, "y_pred": preds})
pred_df.to_csv(
    os.path.join(REPORTS_PATH, f"predictions_{model_type}.csv"),
    index=False
)

# Save model artifact
if SAVE_MODEL:
    os.makedirs(MODELS_PATH, exist_ok=True)
    model_path = os.path.join(MODELS_PATH, f"{model_type}_v1.pkl")
    joblib.dump(model, model_path)
    logger.info(f"Model saved -> {model_path}")

# Save experiment report
save_experiment_report({
    "Date"          : datetime.now().strftime("%Y-%m-%d %H:%M"),
    "Model"         : model_type,
    "Dataset"       : PROCESSED_FILENAME,
    "Features"      : X.shape[1],
    "Train Samples" : len(X_train),
    "Test Samples"  : len(X_test),
    "Accuracy"      : round(acc, 4),
```

```
            "Scaled"        : model_type in NEEDS_SCALING,
        })

if __name__ == "__main__":
    train_pipeline()

import os
import logging
from datetime import datetime
from config import LOGS_PATH, REPORTS_PATH

def create_all_folders():
    """Create all required project directories if they don't exist."""
    from config import (PROCESSED_DATA_PATH, INTERIM_DATA_PATH,
                        MODELS_PATH, SCALERS_PATH, REPORTS_PATH, LOGS_PATH)
    folders = [PROCESSED_DATA_PATH, INTERIM_DATA_PATH,
               MODELS_PATH, SCALERS_PATH, REPORTS_PATH, LOGS_PATH]
    for folder in folders:
        os.makedirs(folder, exist_ok=True)

def get_logger(name: str) -> logging.Logger:
    """Returns a logger that writes to both console and a log file."""
    os.makedirs(LOGS_PATH, exist_ok=True)
    log_file = os.path.join(LOGS_PATH, f"{name}_{datetime.now().strftime('%Y%m%d')}.log")

    logger = logging.getLogger(name)
    logger.setLevel(logging.INFO)

    if not logger.handlers:
        fmt = logging.Formatter("%(asctime)s | %(levelname)s | %(message)s")

        fh = logging.FileHandler(log_file, encoding="utf-8")
        fh.setFormatter(fmt)

        ch = logging.StreamHandler()
        ch.setFormatter(fmt)

        logger.addHandler(fh)
        logger.addHandler(ch)

    return logger
```

```
def save_experiment_report(report: dict, filename: str = None):
    """Save experiment results as a text file in artifacts/reports/."""
    os.makedirs(REPORTS_PATH, exist_ok=True)
    if filename is None:
        filename = f"experiment_{datetime.now().strftime('%Y%m%d_%H%M%S')}.txt"

    filepath = os.path.join(REPORTS_PATH, filename)
    with open(filepath, "w") as f:
        for key, value in report.items():
            f.write(f"{key}: {value}\n")
    print(f"Report saved → {filepath}")
```





```
def run_drift_experiment():

    # Build per-batch per-model accuracy dict
    acc_table: dict = {b: {} for b in batch_order}
    for r in eval_results:
        acc_table[r["batch"]][r["model"]] = r["accuracy"]

    models_seen = CROSS_EVAL_MODELS
    col_w       = 12

    header = f"{'Batch':<52}" + "".join(f"{m:>{col_w}}" for m in models_seen)
    logger.info(header)
    logger.info("-" * len(header))

    for batch in batch_order:
        row = f"{batch:<52}"
        for m in models_seen:
            val = acc_table[batch].get(m, "N/A")
```

```
2026-04-04 18:29:47,713 | INFO |   [Friday-WorkingHours-Afternoon-] kept 225,711 rows, removed 34 NaN/Inf rows
2026-04-04 18:29:47,770 | INFO | Stage 1 complete: 2,827,876 rows written to interim/
2026-04-04 18:29:47,771 | INFO | === Stage 2: Fitting feature selector on Monday ===
2026-04-04 18:29:47,771 | INFO | === Stage 2: Fitting feature selector on Monday ===
Traceback (most recent call last): …
=== Training: xgboost ===
2026-04-04 19:22:49,324 | INFO | Accuracy: 0.9979
2026-04-04 19:22:49,325 | INFO |
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     16057
           1       1.00      0.99      0.99      3943

    accuracy                           1.00     20000
   macro avg       1.00      1.00      1.00     20000
weighted avg       1.00      1.00      1.00     20000

2026-04-04 19:22:49,326 | INFO | Confusion Matrix:
[[16040    17]
 [   26  3917]]
2026-04-04 19:22:49,355 | INFO | Model saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\model\xgboost_v1.pkl
Report saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\experiment_20260404_192249.txt
2026-04-04 19:22:49,360 | INFO |
=== Training: logistic ===
D:\MUJ\Projects\data-drift-prediction-in-ml-models\AshDrift\Lib\site-packages\sklearn\linear_model\_logistic.py:1184: FutureWarning: 'n_jobs' has no effe
t since 1.8 and will be removed in 1.10. You provided 'n_jobs=-1', please leave it unspecified.
  warnings.warn(msg, category=FutureWarning)
2026-04-04 19:22:51,162 | INFO | Accuracy: 0.9383
2026-04-04 19:22:51,162 | INFO |
              precision    recall  f1-score   support

           0       0.97      0.96      0.96     16057
           1       0.83      0.86      0.85      3943

    accuracy                           0.94     20000
   macro avg       0.90      0.91      0.90     20000
weighted avg       0.94      0.94      0.94     20000
```

```
2026-04-04 18:29:47,713 | INFO |   [Friday-WorkingHours-Afternoon-] kept 225,711 rows, removed 34 NaN/Inf rows
2026-04-04 18:29:47,770 | INFO | Stage 1 complete: 2,827,876 rows written to interim/
2026-04-04 18:29:47,771 | INFO | === Stage 2: Fitting feature selector on Monday ===
2026-04-04 18:29:47,771 | INFO | === Stage 2: Fitting feature selector on Monday ===
Traceback (most recent call last): …
2026-04-04 19:22:51,164 | INFO | Confusion Matrix:
[[15300   677]
 [  557  3386]]
2026-04-04 19:22:51,196 | INFO | Model saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\model\logistic_v1.pkl
Report saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\experiment_20260404_192251.txt
2026-04-04 19:22:51,198 | INFO |
=== Training: decision_tree ===
2026-04-04 19:22:53,331 | INFO | Accuracy: 0.9929
2026-04-04 19:22:53,332 | INFO |
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     16057
           1       0.98      0.98      0.98      3943

    accuracy                           0.99     20000
   macro avg       0.99      0.99      0.99     20000
weighted avg       0.99      0.99      0.99     20000

2026-04-04 19:22:53,333 | INFO | Confusion Matrix:
[[15982    75]
 [   66  3877]]
2026-04-04 19:22:53,348 | INFO | Model saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\model\decision_tree_v1.pkl
Report saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\experiment_20260404_192253.txt
2026-04-04 19:22:53,351 | INFO |
=== Training: svm ===
2026-04-04 19:23:08,079 | INFO | Accuracy: 0.9359
2026-04-04 19:23:08,080 | INFO |
              precision    recall  f1-score   support

           0       0.96      0.96      0.96     16057
           1       0.83      0.85      0.84      3943

    accuracy                           0.94     20000
```

Untitled (Workspace)

EXPLORER

UNTITLED (WORKSPACE)
- Autonomous-Code-Reviewer-Security-Auditor
- Miscellaneous
- midterm_hostedPPT
- CarbonEmissionPredictionRNN-NinOA
- data-drift-prediction-in-ml-models
  - artifacts
  - AshDrift
  - data
    - interim
    - processed
      - processed_data_v1.csv
    - processedv1
    - raw
  - deleteLater
  - docs
  - logs
    - v1
    - drift_20260404.log
    - feature_engineering_20260404.log
    - preprocessing_20260404.log
    - testdrift_20260404.log
    - train_20260404.log
  - notebooks
    - .ipynb_checkpoints
    - drift_analysis.ipynb M
  - src
    - __pycache__
    - config.py M
    - drift_predictor.py
    - drift.py
    - feature_engineering.py M
    - main.py
    - models.py
    - preprocessing.py M
    - shap_analysis.py
    - testdrift.py
    - train.py
    - utils.py
  - .gitignore
  - README.md
- natural-language-processing-concepts
- jenkins-template-engine
- pipeline-v5

OUTLINE

TIMELINE

preprocessing.py M | config.py M | train.py | feature_engineering.py M | testdrift.py | models.py

data-drift-prediction-in-ml-models > src > testdrift.py > run_drift_experiment

```python
42  def run_drift_experiment():
103
104     # Build per-batch per-model accuracy dict
105     acc_table: dict = {b: {} for b in batch_order}
106     for r in eval_results:
107         acc_table[r["batch"]][r["model"]] = r["accuracy"]
108
109     models_seen = CROSS_EVAL_MODELS
110     col_w       = 12          You, 4 hours ago • v1.2: add new features to pipeline
111
112     header = f"{'Batch':<52}" + "".join(f"{m:>{col_w}}" for m in models_seen)
113     logger.info(header)
114     logger.info("-" * len(header))
115
116     for batch in batch_order:
117         row = f"{batch:<52}"
118         for m in models_seen:
119             val = acc_table[batch].get(m, "N/A")
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | GITLENS

```
2026-04-04 18:29:47,713 | INFO |   [Friday-WorkingHours-Afternoon-] kept 225,711 rows, removed 34 NaN/Inf rows
2026-04-04 18:29:47,770 | INFO | Stage 1 complete: 2,827,876 rows written to interim/
2026-04-04 18:29:47,771 | INFO | === Stage 2: Fitting feature selector on Monday ===
2026-04-04 18:29:47,771 | INFO | === Stage 2: Fitting feature selector on Monday ===
Traceback (most recent call last): …
2026-04-04 19:22:53,332 | INFO |
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     16057
           1       0.98      0.98      0.98      3943

    accuracy                           0.99     20000
   macro avg       0.99      0.99      0.99     20000
weighted avg       0.99      0.99      0.99     20000

2026-04-04 19:22:53,333 | INFO | Confusion Matrix:
[[15982    75]
 [   66  3877]]
2026-04-04 19:22:53,348 | INFO | Model saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\model\decision_tree_v1.pkl
Report saved → D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\experiment_20260404_192253.txt
2026-04-04 19:22:53,351 | INFO |
=== Training: svm ===
2026-04-04 19:23:08,079 | INFO | Accuracy: 0.9359
2026-04-04 19:23:08,080 | INFO |
              precision    recall  f1-score   support

           0       0.96      0.96      0.96     16057
           1       0.83      0.85      0.84      3943

    accuracy                           0.94     20000
   macro avg       0.90      0.90      0.90     20000
weighted avg       0.94      0.94      0.94     20000

2026-04-04 19:23:08,081 | INFO | Confusion Matrix:
[[15378   679]
 [  602  3341]]
2026-04-04 19:23:08,109 | INFO | Model saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\model\svm_v1.pkl
Report saved → D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\experiment_20260404_192308.txt
```

data-drift-prediction-in-ml-models | v1.0.2* | 01 1↑ | 0 ⚠ 0 | Git Graph | You, 4 hours ago | Anwesha-code (4 hours ago) | Ln 110, Col 17 | Spaces: 4 | UTF-8 | CRLF | Python | AshDrift (3.13.12)

Untitled (Workspace)

EXPLORER

UNTITLED (WORKSPACE)
- Autonomous-Code-Reviewer-Security-Auditor
- Miscellaneous
- midterm_hostedPPT
- CarbonEmissionPredictionRNN-NinOA
- data-drift-prediction-in-ml-models
  - artifacts
  - AshDrift
  - data
    - interim
    - processed
      - processed_data_v1.csv
    - processedv1
    - raw
  - deleteLater
  - docs
  - logs
    - v1
    - drift_20260404.log
    - feature_engineering_20260404.log
    - preprocessing_20260404.log
    - testdrift_20260404.log
    - train_20260404.log
  - notebooks
    - .ipynb_checkpoints
    - drift_analysis.ipynb M
  - src
    - __pycache__
    - config.py M
    - drift_predictor.py
    - drift.py
    - feature_engineering.py M
    - main.py
    - models.py
    - preprocessing.py M
    - shap_analysis.py
    - testdrift.py M
    - train.py
    - utils.py
  - .gitignore
  - README.md
- natural-language-processing-concepts
- jenkins-template-engine
- pipeline-v5

OUTLINE

TIMELINE

preprocessing.py M | config.py M | train.py | feature_engineering.py M | testdrift.py M | models.

data-drift-prediction-in-ml-models > src > testdrift.py > run_drift_experiment

```python
42  def run_drift_experiment():
123
124     header = f"{'Batch':<52}" + "".join(f"{m:>{col_w}}" for m in models_seen)
125     logger.info(header)
126     logger.info("-" * len(header))
127
128     for batch in batch_order:
129         row = f"{batch:<52}"
130         for m in models_seen:
131             val = acc_table[batch].get(m, "N/A")
132             row += f"{str(val):>{col_w}}"
133         logger.info(row)
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | GITLENS

```
    )
     ^^
    )
    ^
  File "D:\MUJ\Projects\data-drift-prediction-in-ml-models\AshDrift\.LIB\site-packages\pandas\core\internals\blocks.py", line 1015, in take_nd …
2026-04-04 19:30:06,895 | INFO | Batch [Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv]: 28623 rows
2026-04-04 19:30:06,916 | INFO | Batch [Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv]: 22591 rows
2026-04-04 19:30:06,916 | INFO | Total batches found: 8
2026-04-04 19:30:06,918 | INFO |
--- Label Distribution per Batch ---
2026-04-04 19:30:06,942 | INFO | [Monday-WorkingHours.pcap_ISCX.csv] Label distribution:
Label
0    1.0
Name: proportion, dtype: float64

2026-04-04 19:30:06,945 | INFO | [Tuesday-WorkingHours.pcap_ISCX.csv] Label distribution:
Label
0    0.96893
1    0.03107
Name: proportion, dtype: float64

2026-04-04 19:30:06,947 | INFO | [Wednesday-workingHours.pcap_ISCX.csv] Label distribution:
Label
0    0.634825
1    0.365175
Name: proportion, dtype: float64

2026-04-04 19:30:06,948 | INFO | [Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv] Label distribution:
Label
0    0.988228
1    0.011772
Name: proportion, dtype: float64

2026-04-04 19:30:06,950 | INFO | [Thursday-WorkingHours-Afternoon-Infiltration.pcap_ISCX.csv] Label distribution:
Label
0    0.999861
1    0.000139
Name: proportion, dtype: float64

2026-04-04 19:30:06,951 | INFO | [Friday-WorkingHours-Morning.pcap_ISCX.csv] Label distribution:
Label
0    0.989575
1    0.010425
Name: proportion, dtype: float64
```

data-drift-prediction-in-ml-models | v1.0.2* | 01 1↑ | 0 ⚠ 0 | Git Graph | You, 2 days ago via PR #3 | Anwesha-code (2 days ago) | Ln 82, Col 1 | Spaces: 4 | UTF-8 | CRLF | Python | AshDrift (3.13.12)

Untitled (Workspace)

preprocessing.py M | config.py M | train.py | feature_engineering.py M | testdrift.py M | models

data-drift-prediction-in-ml-models > src > testdrift.py > run_drift_experiment

```python
def run_drift_experiment():

    header = f"{'Batch':<52}" + "".join(f"{m:>{col_w}}" for m in models_seen)
    logger.info(header)
    logger.info("-" * len(header))

    for batch in batch_order:
        row = f"{batch:<52}"
        for m in models_seen:
            val = acc_table[batch].get(m, "N/A")
            row += f"{str(val):>{col_w}}"
        logger.info(row)
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | GITLENS

```
    )
     ^^
    )
     ^
  File "D:\MUJ\Projects\data-drift-prediction-in-ml-models\AshDrift\Lib\site-packages\pandas\core\internals\blocks.py", line 1015, in take_nd …

2026-04-04 19:30:06,958 | INFO |
--- Fixed Reference Drift (Monday vs all batches) ---
2026-04-04 19:30:07,603 | INFO | [Fixed] 0 vs 1 | KL=0.5770 | JS=0.0109 | WS=58459.0496 | FeatJS=0.0006 | Tuesday-WorkingHours.pcap_ISCX.csv
2026-04-04 19:30:08,555 | INFO | [Fixed] 0 vs 2 | KL=7.7521 | JS=0.1471 | WS=1841756.3783 | FeatJS=0.0235 | Wednesday-workingHours.pcap_ISCX.csv
2026-04-04 19:30:08,998 | INFO | [Fixed] 0 vs 3 | KL=0.2871 | JS=0.0041 | WS=108933.2041 | FeatJS=0.0013 | Thursday-WorkingHours-Morning-WebAttacks.pcap_I
SCX.csv
2026-04-04 19:30:09,576 | INFO | [Fixed] 0 vs 4 | KL=0.0018 | JS=0.0000 | WS=178856.0537 | FeatJS=0.0027 | Thursday-WorkingHours-Afternoon-Infilteration.p
cap_ISCX.csv
2026-04-04 19:30:10,165 | INFO | [Fixed] 0 vs 5 | KL=0.1821 | JS=0.0036 | WS=114944.2999 | FeatJS=0.0013 | Friday-WorkingHours-Morning.pcap_ISCX.csv
2026-04-04 19:30:10,678 | INFO | [Fixed] 0 vs 6 | KL=12.0876 | JS=0.2469 | WS=390731.0981 | FeatJS=0.0110 | Friday-WorkingHours-Afternoon-PortScan.pcap_IS
CX.csv
2026-04-04 19:30:11,269 | INFO | [Fixed] 0 vs 7 | KL=12.3652 | JS=0.2540 | WS=984674.1021 | FeatJS=0.0341 | Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.c
sv
2026-04-04 19:30:11,275 | INFO | Drift results saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\drift_scores.json
2026-04-04 19:30:11,276 | INFO |
--- Rolling Reference Window Drift ---
2026-04-04 19:30:11,933 | INFO | [Rolling] 0 -> 1 | KL=0.5770 | JS=0.0109 | WS=58459.0496 | Monday-WorkingHours.pcap_ISCX.csv -> Tuesday-WorkingHours.pcap
_ISCX.csv
2026-04-04 19:30:12,837 | INFO | [Rolling] 1 -> 2 | KL=0.6314 | JS=0.1004 | WS=1845590.1866 | Tuesday-WorkingHours.pcap_ISCX.csv -> Wednesday-workingHours
.pcap_ISCX.csv
2026-04-04 19:30:13,559 | INFO | [Rolling] 2 -> 3 | KL=0.3969 | JS=0.1238 | WS=1811922.2259 | Wednesday-workingHours.pcap_ISCX.csv -> Thursday-WorkingHour
s-Morning-WebAttacks.pcap_ISCX.csv
2026-04-04 19:30:13,902 | INFO | [Rolling] 3 -> 4 | KL=0.0111 | JS=0.0038 | WS=196961.0338 | Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv -> Thu
rsday-WorkingHours-Afternoon-Infilteration.pcap_ISCX.csv
2026-04-04 19:30:14,365 | INFO | [Rolling] 4 -> 5 | KL=0.0348 | JS=0.0033 | WS=259451.7737 | Thursday-WorkingHours-Afternoon-Infilteration.pcap_ISCX.csv -
> Friday-WorkingHours-Morning.pcap_ISCX.csv
2026-04-04 19:30:14,747 | INFO | [Rolling] 5 -> 6 | KL=1.8494 | JS=0.2229 | WS=479789.5165 | Friday-WorkingHours-Morning.pcap_ISCX.csv -> Friday-WorkingHo
urs-Afternoon-PortScan.pcap_ISCX.csv
2026-04-04 19:30:15,142 | INFO | [Rolling] 6 -> 7 | KL=0.0003 | JS=0.0001 | WS=1215287.8146 | Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv -> Frid
ay-WorkingHours-Afternoon-DDos.pcap_ISCX.csv
2026-04-04 19:30:15,146 | INFO | Rolling drift results saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\rolling_drift_scores.
json
2026-04-04 19:30:15,147 | INFO |
--- Multi-Model Cross-Dataset Evaluation ---
Models: ['random_forest', 'xgboost', 'logistic', 'decision_tree', 'svm']
2026-04-04 19:30:16,705 | INFO |
--- Cross-eval model: random_forest ---
[Parallel(n_jobs=-1)]: Using backend ThreadingBackend with 12 concurrent workers.
```

Untitled (Workspace)

preprocessing.py M | config.py M | train.py | feature_engineering.py M | testdrift.py M | models

data-drift-prediction-in-ml-models > src > testdrift.py > run_drift_experiment

```python
def run_drift_experiment():

    header = f"{'Batch':<52}" + "".join(f"{m:>{col_w}}" for m in models_seen)
    logger.info(header)
    logger.info("-" * len(header))

    for batch in batch_order:
        row = f"{batch:<52}"
        for m in models_seen:
            val = acc_table[batch].get(m, "N/A")
            row += f"{str(val):>{col_w}}"
        logger.info(row)
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | GITLENS

```
    )
     ^^
    )
     ^
  File "D:\MUJ\Projects\data-drift-prediction-in-ml-models\AshDrift\Lib\site-packages\pandas\core\internals\blocks.py", line 1015, in take_nd …
Models: ['random_forest', 'xgboost', 'logistic', 'decision_tree', 'svm']
2026-04-04 19:30:16,705 | INFO |
--- Cross-eval model: random_forest ---
[Parallel(n_jobs=-1)]: Using backend ThreadingBackend with 12 concurrent workers.
[Parallel(n_jobs=-1)]: Done  26 tasks      | elapsed:    0.2s
[Parallel(n_jobs=-1)]: Done 100 out of 100 | elapsed:    0.4s finished
[Parallel(n_jobs=12)]: Using backend ThreadingBackend with 12 concurrent workers.
[Parallel(n_jobs=12)]: Done  26 tasks      | elapsed:    0.0s
[Parallel(n_jobs=12)]: Done 100 out of 100 | elapsed:    0.0s finished
2026-04-04 19:30:17,318 | INFO |   Self-accuracy: 1.0000
[Parallel(n_jobs=12)]: Using backend ThreadingBackend with 12 concurrent workers.
[Parallel(n_jobs=12)]: Done  26 tasks      | elapsed:    0.0s
[Parallel(n_jobs=12)]: Done 100 out of 100 | elapsed:    0.0s finished
2026-04-04 19:30:17,394 | INFO |   Batch 1 [Tuesday-WorkingHours.pcap_ISCX.csv]: 0.9689
[Parallel(n_jobs=12)]: Using backend ThreadingBackend with 12 concurrent workers.
[Parallel(n_jobs=12)]: Done  26 tasks      | elapsed:    0.0s
[Parallel(n_jobs=12)]: Done 100 out of 100 | elapsed:    0.0s finished
2026-04-04 19:30:17,470 | INFO |   Batch 2 [Wednesday-workingHours.pcap_ISCX.cs]: 0.6348
[Parallel(n_jobs=12)]: Using backend ThreadingBackend with 12 concurrent workers.
[Parallel(n_jobs=12)]: Done  26 tasks      | elapsed:    0.0s
[Parallel(n_jobs=12)]: Done 100 out of 100 | elapsed:    0.0s finished
2026-04-04 19:30:17,550 | INFO |   Batch 3 [Thursday-WorkingHours-Morning-WebAt]: 0.9882
[Parallel(n_jobs=12)]: Using backend ThreadingBackend with 12 concurrent workers.
[Parallel(n_jobs=12)]: Done  26 tasks      | elapsed:    0.0s
[Parallel(n_jobs=12)]: Done 100 out of 100 | elapsed:    0.0s finished
2026-04-04 19:30:17,633 | INFO |   Batch 4 [Thursday-WorkingHours-Afternoon-Inf]: 0.9999
[Parallel(n_jobs=12)]: Using backend ThreadingBackend with 12 concurrent workers.
[Parallel(n_jobs=12)]: Done  26 tasks      | elapsed:    0.1s
[Parallel(n_jobs=12)]: Done 100 out of 100 | elapsed:    0.1s finished
2026-04-04 19:30:17,901 | INFO |   Batch 5 [Friday-WorkingHours-Morning.pcap_IS]: 0.9896
[Parallel(n_jobs=12)]: Using backend ThreadingBackend with 12 concurrent workers.
[Parallel(n_jobs=12)]: Done  26 tasks      | elapsed:    0.0s
[Parallel(n_jobs=12)]: Done 100 out of 100 | elapsed:    0.0s finished
2026-04-04 19:30:17,983 | INFO |   Batch 6 [Friday-WorkingHours-Afternoon-PortS]: 0.4452
[Parallel(n_jobs=12)]: Using backend ThreadingBackend with 12 concurrent workers.
[Parallel(n_jobs=12)]: Done  26 tasks      | elapsed:    0.0s
[Parallel(n_jobs=12)]: Done 100 out of 100 | elapsed:    0.0s finished
2026-04-04 19:30:18,133 | INFO |   Batch 7 [Friday-WorkingHours-Afternoon-DDos.]: 0.4333
2026-04-04 19:30:18,134 | INFO |
```

Untitled (Workspace)

main.py | preprocessing.py M | config.py M | train.py | feature_engineering.py M | testdrift.py M | drift.py M | models.py

data-drift-prediction-in-ml-models > src > drift.py > ...

```python
def check_drift_alerts(cross_eval_results: list,
                       pass

        alert = {
            "batch"              : batch,
            "average_accuracy"   : round(avg_acc, 4),
            "threshold"          : DRIFT_ALERT_THRESHOLD,
            "per_model_accuracy" : batch_per_model[batch],
            "label_kl_divergence": kl,
            "label_js_divergence": js,
            "predicted_accuracy" : predicted,
            "alert"              : "DRIFT DETECTED — avg accuracy below threshold",
        }
        alerts.append(alert)
        logger.warning(
            f"DRIFT ALERT | {batch[:40]} | "
            f"avg_acc={avg_acc:.4f} < {DRIFT_ALERT_THRESHOLD}"
        )

    if alerts:
        path = os.path.join(REPORTS_PATH, "drift_alerts.json")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
  File "D:\MLO\Projects\data-drift-prediction-in-ml-models\AshDrift\Lib\site-packages\sklearn\linear_model\_logistic.py", line 1243, in fit
    raise ValueError(
    ...<3 lines>...
    ~~~~~~~~~~~~~~~~^^
  File "D:\MLO\Projects\data-drift-prediction-in-ml-models\src\testdrift.py", line 98, in run_drift_experiment …
2026-04-04 21:13:25,527 | INFO |
--- Label Distribution per Batch ---
2026-04-04 21:13:25,640 | INFO | [Monday-WorkingHours.pcap_ISCX.csv] Label distribution:
Label
0   1.0
Name: proportion, dtype: float64

2026-04-04 21:13:25,646 | INFO | [Tuesday-WorkingHours.pcap_ISCX.csv] Label distribution:
Label
0    0.96893
1    0.03107
Name: proportion, dtype: float64

2026-04-04 21:13:25,655 | INFO | [Wednesday-workingHours.pcap_ISCX.csv] Label distribution:
Label
0    0.634825
1    0.365175
Name: proportion, dtype: float64

2026-04-04 21:13:25,661 | INFO | [Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv] Label distribution:
Label
0    0.988228
1    0.011772
Name: proportion, dtype: float64

2026-04-04 21:13:25,666 | INFO | [Thursday-WorkingHours-Afternoon-InFilteration.pcap_ISCX.csv] Label distribution:
Label
0    0.999861
1    0.000139
Name: proportion, dtype: float64

2026-04-04 21:13:25,669 | INFO | [Friday-WorkingHours-Morning.pcap_ISCX.csv] Label distribution:
Label
0    0.989575
1    0.010425
Name: proportion, dtype: float64

2026-04-04 21:13:25,683 | INFO | [Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv] Label distribution:
Label
0    0.445201
1    0.554799
```

Untitled (Workspace)

```
  File "D:\MLO\Projects\data-drift-prediction-in-ml-models\AshDrift\Lib\site-packages\sklearn\linear_model\_logistic.py", line 1243, in fit
    raise ValueError(
    ...<3 lines>...
    ~~~~~~~~~~~~~~~~^^
  File "D:\MLO\Projects\data-drift-prediction-in-ml-models\src\testdrift.py", line 98, in run_drift_experiment …
2026-04-04 21:13:25,703 | INFO | [Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv] Label distribution:
Label
0    0.43327
1    0.56673
Name: proportion, dtype: float64

2026-04-04 21:13:25,705 | INFO |
--- Fixed Reference Drift (Monday vs all batches) ---
2026-04-04 21:13:26,759 | INFO | [Fixed] 0 vs 1 | KL=0.5770 | JS=0.0109 | WS=58459.0496 | FeatJS=0.0006 | Tuesday-WorkingHours.pcap_ISCX.csv
2026-04-04 21:13:27,750 | INFO | [Fixed] 0 vs 2 | KL=7.7521 | JS=0.3471 | WS=1841756.3783 | FeatJS=0.0235 | Wednesday-workingHours.pcap_ISCX.csv
2026-04-04 21:13:28,263 | INFO | [Fixed] 0 vs 3 | KL=0.3073 | JS=0.0041 | WS=108933.2041 | FeatJS=0.0013 | Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv
2026-04-04 21:13:28,788 | INFO | [Fixed] 0 vs 4 | KL=0.0018 | JS=0.0000 | WS=1708856.0537 | FeatJS=0.0027 | Thursday-WorkingHours-Afternoon-Infilteration.pcap_ISCX.csv
2026-04-04 21:13:29,156 | INFO | [Fixed] 0 vs 5 | KL=0.1821 | JS=0.0036 | WS=114944.2999 | FeatJS=0.0013 | Friday-WorkingHours-Morning.pcap_ISCX.csv
2026-04-04 21:13:29,959 | INFO | [Fixed] 0 vs 6 | KL=12.0876 | JS=0.2469 | WS=390731.6981 | FeatJS=0.0130 | Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv
2026-04-04 21:13:30,640 | INFO | [Fixed] 0 vs 7 | KL=12.3652 | JS=0.2540 | WS=980674.1021 | FeatJS=0.0341 | Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv
2026-04-04 21:13:30,668 | INFO | Drift results saved -> D:\MLO\Projects\data-drift-prediction-in-ml-models\artifacts\reports\drift_scores.json
2026-04-04 21:13:30,669 | INFO |
--- Rolling Reference Window Drift ---
2026-04-04 21:13:31,536 | INFO | [Rolling] 0 -> 1 | KL=0.5770 | JS=0.0109 | WS=58459.0496 | Monday-WorkingHours.pcap_ISCX.csv -> Tuesday-WorkingHours.pcap_ISCX.csv
2026-04-04 21:13:32,503 | INFO | [Rolling] 1 -> 2 | KL=0.6314 | JS=0.1004 | WS=1845590.1866 | Tuesday-WorkingHours.pcap_ISCX.csv -> Wednesday-workingHours.pcap_ISCX.csv
2026-04-04 21:13:33,698 | INFO | [Rolling] 2 -> 3 | KL=0.3969 | JS=0.1238 | WS=1811922.2259 | Wednesday-workingHours.pcap_ISCX.csv -> Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.c
2026-04-04 21:13:34,094 | INFO | [Rolling] 3 -> 4 | KL=0.0111 | JS=0.0038 | WS=190901.0938 | Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv -> Thursday-WorkingHours-Afternoon-Inf
ion.pcap_ISCX.csv
2026-04-04 21:13:34,680 | INFO | [Rolling] 4 -> 5 | KL=0.0348 | JS=0.0033 | WS=259451.7737 | Thursday-WorkingHours-Afternoon-Infilteration.pcap_ISCX.csv -> Friday-WorkingHours-Morning.pc
.csv
2026-04-04 21:13:34,957 | INFO | [Rolling] 5 -> 6 | KL=1.8494 | JS=0.2229 | WS=479789.5165 | Friday-WorkingHours-Morning.pcap_ISCX.csv -> Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX
2026-04-04 21:13:35,361 | INFO | [Rolling] 6 -> 7 | KL=0.0003 | JS=0.0001 | WS=1215287.8146 | Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv -> Friday-WorkingHours-Afternoon-DDos.p
X.csv
2026-04-04 21:13:35,363 | INFO | Rolling drift results saved -> D:\MLO\Projects\data-drift-prediction-in-ml-models\artifacts\reports\rolling_drift_scores.json
2026-04-04 21:13:35,364 | INFO |
--- Multi-Model Cross-Dataset Evaluation ---
Models: ['random_forest', 'xgboost', 'logistic', 'decision_tree', 'svm']
2026-04-04 21:13:37,311 | WARNING | Reference batch 'Monday-WorkingHours.pcap_ISCX.csv' contains only 1 class(es) — solvers that require 2+ classes (logistic, svm) will use a DummyClassi
llback.
2026-04-04 21:13:37,314 | INFO |
--- Cross-eval model: random_forest ---
[Parallel(n_jobs=-1)]: Using backend ThreadingBackend with 12 concurrent workers.
[Parallel(n_jobs=-1)]: Done  26 tasks      | elapsed:    0.4s
[Parallel(n_jobs=-1)]: Done 100 out of 100 | elapsed:    0.8s finished
[Parallel(n_jobs=12)]: Using backend ThreadingBackend with 12 concurrent workers.
[Parallel(n_jobs=12)]: Done  26 tasks      | elapsed:    0.0s
```

```
def check_drift_alerts(cross_eval_results: list,
                       ...):
    ...
    alert = {
        "batch"             : batch,
        "average_accuracy"  : round(avg_acc, 4),
        "threshold"         : DRIFT_ALERT_THRESHOLD,
        "per_model_accuracy" : batch_per_model[batch],
        "label_kl_divergence": kl,
        "label_js_divergence": js,
        "predicted_accuracy" : predicted,
        "alert"             : "DRIFT DETECTED — avg accuracy below threshold",
    }
    alerts.append(alert)
    logger.warning(
        f"DRIFT ALERT | {batch:14s} | "
        f"avg_acc={avg_acc:.4f} < {DRIFT_ALERT_THRESHOLD}"
    )
if alerts:
    path = os.path.join(REPORTS_PATH, "drift_alerts.json")
```

```
  File "D:\MUJ\Projects\data-drift-prediction-in-ml-models\AshDrift\Lib\site-packages\sklearn\linear_model\_logistic.py", line 1243, in fit
    raise ValueError(
    ...<3 lines>...
  File "D:\MUJ\Projects\data-drift-prediction-in-ml-models\src\testdrift.py", line 90, in run_drift_experiment …
2026-04-04 21:13:38,578 | INFO |   Batch 1 [Tuesday-WorkingHours.pcap_ISCX.csv]: 0.9689
[Parallel(n_jobs=12)]: Using backend ThreadingBackend with 12 concurrent workers.
[Parallel(n_jobs=12)]: Done  26 tasks      | elapsed:   0.0s
[Parallel(n_jobs=12)]: Done 100 out of 100 | elapsed:   0.1s finished
2026-04-04 21:13:38,939 | INFO |   Batch 2 [Wednesday-workingHours.pcap_ISCX.cs]: 0.6348
[Parallel(n_jobs=12)]: Using backend ThreadingBackend with 12 concurrent workers.
[Parallel(n_jobs=12)]: Done  26 tasks      | elapsed:   0.0s
[Parallel(n_jobs=12)]: Done 100 out of 100 | elapsed:   0.0s finished
2026-04-04 21:13:39,186 | INFO |   Batch 3 [Thursday-WorkingHours-Morning-WebAt]: 0.9882
[Parallel(n_jobs=12)]: Using backend ThreadingBackend with 12 concurrent workers.
[Parallel(n_jobs=12)]: Done  26 tasks      | elapsed:   0.0s
[Parallel(n_jobs=12)]: Done 100 out of 100 | elapsed:   0.0s finished
2026-04-04 21:13:39,348 | INFO |   Batch 4 [Thursday-WorkingHours-Afternoon-Inf]: 0.9999
[Parallel(n_jobs=12)]: Using backend ThreadingBackend with 12 concurrent workers.
[Parallel(n_jobs=12)]: Done  26 tasks      | elapsed:   0.3s
[Parallel(n_jobs=12)]: Done 100 out of 100 | elapsed:   0.4s finished
2026-04-04 21:13:39,888 | INFO |   Batch 5 [Friday-WorkingHours-Morning.pcap_IS]: 0.9896
[Parallel(n_jobs=12)]: Using backend ThreadingBackend with 12 concurrent workers.
[Parallel(n_jobs=12)]: Done  26 tasks      | elapsed:   0.0s
[Parallel(n_jobs=12)]: Done 100 out of 100 | elapsed:   0.0s finished
2026-04-04 21:13:40,162 | INFO |   Batch 6 [Friday-WorkingHours-Afternoon-PortS]: 0.4452
[Parallel(n_jobs=12)]: Using backend ThreadingBackend with 12 concurrent workers.
[Parallel(n_jobs=12)]: Done  26 tasks      | elapsed:   0.0s
[Parallel(n_jobs=12)]: Done 100 out of 100 | elapsed:   0.1s finished
2026-04-04 21:13:40,171 | INFO |   Batch 7 [Friday-WorkingHours-Afternoon-DDos.]: 0.4333
2026-04-04 21:13:40,172 | INFO |
--- Cross-eval model: xgboost ---
2026-04-04 21:13:41,060 | INFO |   Self-accuracy: 1.0000
2026-04-04 21:13:41,097 | INFO |   Batch 1 [Tuesday-WorkingHours.pcap_ISCX.csv]: 0.9689
2026-04-04 21:13:41,149 | INFO |   Batch 2 [Wednesday-workingHours.pcap_ISCX.cs]: 0.6348
2026-04-04 21:13:41,178 | INFO |   Batch 3 [Thursday-WorkingHours-Morning-WebAt]: 0.9882
2026-04-04 21:13:41,203 | INFO |   Batch 4 [Thursday-WorkingHours-Afternoon-Inf]: 0.9999
2026-04-04 21:13:41,228 | INFO |   Batch 5 [Friday-WorkingHours-Morning.pcap_IS]: 0.9896
2026-04-04 21:13:41,250 | INFO |   Batch 6 [Friday-WorkingHours-Afternoon-PortS]: 0.4452
2026-04-04 21:13:41,269 | INFO |   Batch 7 [Friday-WorkingHours-Afternoon-DDos.]: 0.4333
2026-04-04 21:13:41,269 | INFO |
--- Cross-eval model: logistic ---
2026-04-04 21:13:41,298 | WARNING |   [logistic] Reference is single-class — using DummyClassifier (always predicts majority class).
2026-04-04 21:13:41,306 | INFO |   Self-accuracy: 1.0000
2026-04-04 21:13:41,375 | INFO |   Batch 1 [Tuesday-WorkingHours.pcap_ISCX.csv]: 0.9689
2026-04-04 21:13:41,458 | INFO |   Batch 2 [Wednesday-workingHours.pcap_ISCX.cs]: 0.6348
```

```
  File "D:\MUJ\Projects\data-drift-prediction-in-ml-models\AshDrift\Lib\site-packages\sklearn\linear_model\_logistic.py", line 1243, in fit
    raise ValueError(
    ...<3 lines>...
  File "D:\MUJ\Projects\data-drift-prediction-in-ml-models\src\testdrift.py", line 90, in run_drift_experiment …
2026-04-04 21:13:42,528 | INFO |   Batch 3 [Thursday-WorkingHours-Morning-WebAt]: 0.9882
2026-04-04 21:13:42,584 | INFO |   Batch 4 [Thursday-WorkingHours-Afternoon-Inf]: 0.9999
2026-04-04 21:13:42,626 | INFO |   Batch 5 [Friday-WorkingHours-Morning.pcap_IS]: 0.9896
2026-04-04 21:13:42,691 | INFO |   Batch 6 [Friday-WorkingHours-Afternoon-PortS]: 0.4452
2026-04-04 21:13:42,784 | INFO |   Batch 7 [Friday-WorkingHours-Afternoon-DDos.]: 0.4333
2026-04-04 21:13:42,803 | INFO | Cross-eval results saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\cross_eval_accuracy.json
2026-04-04 21:13:42,804 | INFO |
--- Checking Drift Alerts ---
2026-04-04 21:13:42,809 | WARNING | DRIFT ALERT | Wednesday-workingHours.pcap_ISCX.csv | avg_acc=0.6348 < 0.8
2026-04-04 21:13:42,811 | WARNING | DRIFT ALERT | Friday-WorkingHours-Afternoon-PortScan.p | avg_acc=0.4452 < 0.8
2026-04-04 21:13:42,814 | WARNING | DRIFT ALERT | Friday-WorkingHours-Afternoon-DDos.pcap_ | avg_acc=0.4333 < 0.8
2026-04-04 21:13:42,823 | INFO | Alerts saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\drift_alerts.json
2026-04-04 21:13:42,825 | INFO |
=== CROSS-DATASET EVALUATION SUMMARY ===
2026-04-04 21:13:42,827 | INFO | Batch                                          random_forest   xgboost   logisticdecision_true       sum  2026-04-04 21:13:42,839 | INFO | Wednesday-
2026-04-04 21:13:42,839 | INFO | Wednesday-workingHours.pcap_ISCX.csv                  0.6348    0.6348    0.6348    0.6348    0.6348
2026-04-04 21:13:42,841 | INFO | Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv   0.9882    0.9882    0.9882    0.9882    0.9882
2026-04-04 21:13:42,841 | INFO | Thursday-WorkingHours-Afternoon-Infiltration.pcap_ISCX.csv   0.9999    0.9999    0.9999    0.9999    0.9999
2026-04-04 21:13:42,841 | INFO | Friday-WorkingHours-Morning.pcap_ISCX.csv             0.9896    0.9896    0.9896    0.9896    0.9896
2026-04-04 21:13:42,842 | INFO | Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv  0.4452    0.4452    0.4452    0.4452    0.4452
2026-04-04 21:13:42,842 | INFO | Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv      0.4333    0.4333    0.4333    0.4333    0.4333
2026-04-04 21:13:42,842 | INFO |
Drift scores        -> artifacts/reports/drift_scores.json
2026-04-04 21:13:42,843 | INFO | Rolling drift       -> artifacts/reports/rolling_drift_scores.json
2026-04-04 21:13:42,844 | INFO | Cross-eval accuracy  -> artifacts/reports/cross_eval_accuracy.json
2026-04-04 21:13:42,845 | INFO | Drift alerts        -> artifacts/reports/drift_alerts.json  (3 alerts)
2026-04-04 21:13:42,846 | INFO | === Drift Experiment Complete ===
(AshDrift) PS D:\MUJ\Projects\data-drift-prediction-in-ml-models\src>
```

Untitled (Workspace)

main.py | config.py M | shap_analysis.py M | utils.py | models.py

data-drift-prediction-in-ml-models > src > shap_analysis.py > run_shap_for_model

```python
def run_shap_for_model(model_type, ref_batch, drifted_batch, ref_name, drifted_name):
        if isinstance(sv_ref, list):
            # Older SHAP versions return a list
            sv_ref    = sv_ref[1]
            sv_drifted = sv_drifted[1]
        elif len(sv_ref.shape) == 3:
            # Newer SHAP versions return a 3D array (samples, features, classes)
            sv_ref    = sv_ref[:, :, 1]
            sv_drifted = sv_drifted[:, :, 1]

    except Exception as e:
        logger.error(f"SHAP failed for {model_type}: {e}")
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | GITLENS

```
PS D:\MUJ\Projects\CarbonEmissionPredictionRNN-NinOA> cd D:\MUJ\Projects\data-drift-prediction-in-ml-models\src
PS D:\MUJ\Projects\data-drift-prediction-in-ml-models\src> python shap_analysis.py
2026-04-04 22:04:13,266 | INFO | === SHAP Analysis Started ===
2026-04-04 22:04:13,272 | INFO | Loading: D:\MUJ\Projects\data-drift-prediction-in-ml-models\data\processed\processed_data_v1.csv
2026-04-04 22:04:13,272 | INFO | DEBUG mode: Chunk-loading to prevent memory crash...
2026-04-04 22:04:40,028 | INFO | Reference : Monday-Working@Hours.pcap_ISCX.csv (52939 rows)
2026-04-04 22:04:40,038 | INFO | Drifted   : Friday-Working@Hours-Afternoon-DDos.pcap_ISCX.csv (22591 rows)
2026-04-04 22:04:49,740 | INFO |
--- SHAP: random_forest ---
2026-04-04 22:05:20,237 | INFO | Plot saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\shap_comparison_random_forest.png
2026-04-04 22:05:20,240 | INFO | JSON saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\shap_values_random_forest.json
2026-04-04 22:05:20,240 | INFO |   Top 5 features — reference vs drifted importance:
2026-04-04 22:05:20,242 | INFO |     Destination Port                ref=0.0265  drift=0.0481  delta=+0.0216
2026-04-04 22:05:20,242 | INFO |     Fwd Packet Length Max           ref=0.0206  drift=0.0444  delta=+0.0238
2026-04-04 22:05:20,242 | INFO |     Packet Length Mean              ref=0.0154  drift=0.0315  delta=+0.0161
2026-04-04 22:05:20,242 | INFO |     Bwd Packet Length Std           ref=0.0151  drift=0.0709  delta=+0.0558
2026-04-04 22:05:20,242 | INFO |     Bwd Packet Length Mean          ref=0.0132  drift=0.0556  delta=+0.0424
2026-04-04 22:05:20,242 | INFO |
--- SHAP: xgboost ---
2026-04-04 22:05:21,300 | INFO | Plot saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\shap_comparison_xgboost.png
2026-04-04 22:05:21,302 | INFO | JSON saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\shap_values_xgboost.json
2026-04-04 22:05:21,302 | INFO |   Top 5 features — reference vs drifted importance:
2026-04-04 22:05:21,302 | INFO |     Init_Win_bytes_forward          ref=1.5721  drift=0.7364  delta=-0.8358
2026-04-04 22:05:21,302 | INFO |     Destination Port                ref=1.2514  drift=1.4743  delta=+0.2229
2026-04-04 22:05:21,304 | INFO |     Fwd Packet Length Max           ref=0.9991  drift=0.9372  delta=-0.0619
2026-04-04 22:05:21,305 | INFO |     Bwd Packet Length Min           ref=0.7446  drift=0.3660  delta=-0.3786
2026-04-04 22:05:21,305 | INFO |     Init_Win_bytes_backward         ref=0.6041  drift=0.5600  delta=-0.0442
2026-04-04 22:05:21,306 | INFO |
--- SHAP: decision_tree ---
2026-04-04 22:05:22,115 | INFO | Plot saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\shap_comparison_decision_tree.png
2026-04-04 22:05:22,119 | INFO | JSON saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\shap_values_decision_tree.json
2026-04-04 22:05:22,120 | INFO |   Top 5 features — reference vs drifted importance:
2026-04-04 22:05:22,121 | INFO |     Total Length of Fwd Packets     ref=0.0624  drift=0.0443  delta=-0.0181
2026-04-04 22:05:22,121 | INFO |     Bwd Packet Length Std           ref=0.0579  drift=0.2978  delta=+0.2399
2026-04-04 22:05:22,122 | INFO |     Destination Port                ref=0.0536  drift=0.0526  delta=-0.0011
2026-04-04 22:05:22,123 | INFO |     Flow Bytes/s                    ref=0.0349  drift=0.0735  delta=+0.0386
2026-04-04 22:05:22,124 | INFO |     PSH Flag Count                  ref=0.0289  drift=0.0162  delta=-0.0127
2026-04-04 22:05:22,126 | INFO |
--- SHAP: logistic ---
2026-04-04 22:05:23,042 | INFO | Plot saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\shap_comparison_logistic.png
2026-04-04 22:05:23,044 | INFO | JSON saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\shap_values_logistic.json
2026-04-04 22:05:23,045 | INFO |   Top 5 features — reference vs drifted importance:
2026-04-04 22:05:23,045 | INFO |     Bwd Packet Length Min           ref=2.8267  drift=2.4952  delta=-0.3314
2026-04-04 22:05:23,045 | INFO |     Fwd Packet Length Std           ref=1.9002  drift=5.5785  delta=+3.6783
```

data-drift-prediction-in-ml-models | v1.0.2* | 0↓ 1↑ | Git Graph | You, 7 hours ago | Anwesha-code (7 hours ago) | Ln 139, Col 15 | Spaces: 4 | UTF-8 | CRLF | Python | AshDrift (3.13.12)

Untitled (Workspace)

main.py | config.py M | shap_analysis.py M | utils.py | models.py

data-drift-prediction-in-ml-models > src > shap_analysis.py > run_shap_for_model

```python
def run_shap_for_model(model_type, ref_batch, drifted_batch, ref_name, drifted_name):
        if isinstance(sv_ref, list):
            # Older SHAP versions return a list
            sv_ref    = sv_ref[1]
            sv_drifted = sv_drifted[1]
        elif len(sv_ref.shape) == 3:
            # Newer SHAP versions return a 3D array (samples, features, classes)
            sv_ref    = sv_ref[:, :, 1]
            sv_drifted = sv_drifted[:, :, 1]

    except Exception as e:
        logger.error(f"SHAP failed for {model_type}: {e}")
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | GITLENS

```
PS D:\MUJ\Projects\data-drift-prediction-in-ml-models\src> python shap_analysis.py
2026-04-04 22:05:22,119 | INFO | JSON saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\shap_values_decision_tree.json
2026-04-04 22:05:22,120 | INFO |   Top 5 features — reference vs drifted importance:
2026-04-04 22:05:22,121 | INFO |     Total Length of Fwd Packets     ref=0.0624  drift=0.0443  delta=-0.0181
2026-04-04 22:05:22,121 | INFO |     Bwd Packet Length Std           ref=0.0579  drift=0.2978  delta=+0.2399
2026-04-04 22:05:22,122 | INFO |     Destination Port                ref=0.0536  drift=0.0526  delta=-0.0011
2026-04-04 22:05:22,123 | INFO |     Flow Bytes/s                    ref=0.0349  drift=0.0735  delta=+0.0386
2026-04-04 22:05:22,124 | INFO |     PSH Flag Count                  ref=0.0289  drift=0.0162  delta=-0.0127
2026-04-04 22:05:22,126 | INFO |
--- SHAP: logistic ---
2026-04-04 22:05:23,042 | INFO | Plot saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\shap_comparison_logistic.png
2026-04-04 22:05:23,044 | INFO | JSON saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\shap_values_logistic.json
2026-04-04 22:05:23,045 | INFO |   Top 5 features — reference vs drifted importance:
2026-04-04 22:05:23,045 | INFO |     Bwd Packet Length Min           ref=2.8267  drift=2.4952  delta=-0.3314
2026-04-04 22:05:23,045 | INFO |     Fwd Packet Length Std           ref=1.9002  drift=5.5785  delta=+3.6783
2026-04-04 22:05:23,046 | INFO |     PSH Flag Count                  ref=1.5675  drift=1.7036  delta=+0.1361
2026-04-04 22:05:23,047 | INFO |     min_seg_size_forward            ref=1.2170  drift=1.3017  delta=+0.0853
2026-04-04 22:05:23,047 | INFO |     ACK Flag Count                  ref=1.1958  drift=1.4976  delta=+0.3019
2026-04-04 22:05:23,051 | INFO |
--- SHAP: svm ---
2026-04-04 22:05:23,048 | INFO | Plot saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\shap_comparison_svm.png
2026-04-04 22:05:23,052 | INFO | JSON saved -> D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports\shap_values_svm.json
2026-04-04 22:05:23,053 | INFO |   Top 5 features — reference vs drifted importance:
2026-04-04 22:05:23,053 | INFO |     Max Packet Length               ref=1.4487  drift=6.3124  delta=+4.8637
2026-04-04 22:05:23,054 | INFO |     Bwd Packet Length Max           ref=1.0757  drift=5.6681  delta=+4.5924
2026-04-04 22:05:23,054 | INFO |     PSH Flag Count                  ref=0.7113  drift=0.7752  delta=+0.0639
2026-04-04 22:05:23,055 | INFO |     Fwd Packet Length Std           ref=0.6250  drift=1.8348  delta=+1.2098
2026-04-04 22:05:23,056 | INFO |     ACK Flag Count                  ref=0.5748  drift=0.7189  delta=+0.1469
2026-04-04 22:05:23,056 | INFO |
=== SHAP Analysis Complete ===
2026-04-04 22:05:23,057 | INFO | All outputs in: D:\MUJ\Projects\data-drift-prediction-in-ml-models\artifacts\reports
PS D:\MUJ\Projects\data-drift-prediction-in-ml-models\src>
```